AI ENGINEERING ROADMAP · WEEK 4

$\nabla L(\theta)$

Math for Machine Learning

$P(A|B)$

A complete written course: lessons, hand-worked examples, NumPy code, exercises, your answers, and step-by-step reviews.

01 Vectors, matrices & tensors

02 Dot product & matmul in NumPy

03 Probability & Bayes' theorem

04 Statistics notebook

05 Derivatives & gradients

$w \leftarrow w - \alpha \cdot \partial L / \partial w$

06 Optimization notebook

07 Linear regression from scratch

08 Comparing learning rates

09 One-page cheat sheet

$\sum x_i w_i$

Contents

How This Course Works 4

Course map 5
How to read each module 5

Module 1 · Vectors, Matrices, Tensors & Dimensions 6

1. Vectors 6
2. Matrices 7
3. Tensors 8
4. Dimensions, shape, and rank 10
5. Your turn: solve these by hand 10
6. Review of your answers 11

What you should remember from this lesson 13

Module 2 · Dot Product & Matrix Multiplication with NumPy 14

1. Dot product of two vectors 14
2. Matrix multiplication 15
3. The most important rule: matrix shapes 16
4. Matrix multiplication is not element-wise multiplication 17
5. Why this matters in AI engineering 18
6. Your turn: NumPy practice 18
7. Review of your answers 19

Your progress: what you've mastered 22

Module 3 · Probability, Conditional Probability & Bayes' Theorem 23

Quick reference 23
Exercises 24
Review of your answers 25
Your Week 4 takeaway 29

Module 4 · Statistics Notebook: Mean, Variance, Distributions & Correlation 30

Notebook 1 · Mean, variance, and standard deviation 30
Notebook 2 · Distributions 33
Notebook 3 · Correlation 34
Notebook 4 · Put it all together 36
Quick reference 37
Practice exercises 37
Review of your answers 38
Final assessment 42

Module 5 · Derivatives, Gradients & Partial Derivatives 43

Exercises 43
Review and corrections 44
Final review 49

Module 6 · Optimization Notebook: Loss Functions, Learning Rate & Gradient Descent 50

1. The big picture 50
2. Loss functions 50
3. Gradient descent 51
4. Learning rate: why it matters 51
5. Your Optimization Notebook 52
6. Exercises 54
7. Answer review 56
8. Optimization retest: aiming for 100/100 60

Module 7 · Linear Regression from Scratch 62

1. What is linear regression? 62
2. How does the model learn? 62
3. Implement linear regression with NumPy 63
4. What output should you expect? 64
5. Check your understanding 65
6. Review of your answers 66

Module 8 · Gradient Descent from Scratch: Comparing Learning Rates 67

1. The idea 67
2. Compare three learning rates 67
3. Implement it from scratch 68
4. What should you observe? 69
5. Experiment with it: the learning-rate playground 70
6. Your turn 70
7. Review of your answers 71
8. Next challenge: a more realistic comparison 72
9. Common mistakes 73
10. Recall test: close the notes first 73

Module 9 · Math for Machine Learning: One-Page Cheat Sheet 74

How This Course Works

This course is the complete written record of your Week 4 study sessions: **Math for Machine Learning**. Every lesson, every worked example, every exercise you solved, your own answers, and every review and correction are included, in the order you learned them.

You started Week 4 with this plan for each topic:

1. **Intuition first:** understand the idea in plain English before diving into equations.
2. **Mathematical foundations:** break down the formulas and explain each component.
3. **Python implementation:** use NumPy or simple Python code to see the concept in action.
4. **ML applications:** connect the math to actual algorithms, such as linear regression, neural networks, or dimensionality reduction.
5. **Practice:** solve a small exercise yourself, then review the answer together.

New concepts are connected to things you have already met in your learning, such as MSE, PCA, `fit()`, `transform()`, and embeddings, whenever relevant.

> **One important rule:** we focus on understanding rather than memorizing formulas. As an AI engineer, you need to know *when* and *why* to use the mathematics, not necessarily derive every equation from scratch.

Course map

Module	Topic	Output
1	Vectors, matrices, tensors, dimensions	Hand-worked examples
2	Dot product and matrix multiplication	NumPy implementation
3	Probability, conditional probability, Bayes	Bayes exercises
4	Mean, variance, distributions, correlation	Statistics notebook
5	Derivatives, gradients, partial derivatives	Gradient exercises
6	Loss functions, learning rate, gradient descent	Optimization notebook + retest
7	Linear regression from scratch	NumPy model
8	Gradient descent from scratch	Comparing learning rates
9	Explain the fundamentals without notes	One-page cheat sheet

How to read each module

Each module follows the same rhythm as our sessions:

- **Lesson:** the explanation, formulas, worked examples, and code.
- **Exercises:** the practice questions exactly as they were set.
- **Your answers:** what you submitted, shown in a highlighted box.
- **Review:** the step-by-step check of every answer, with corrections and explanations.

> **Editor's note:** interactive widgets from the original chat (calculators, sliders, drag-the-points charts) cannot exist on paper, so they have been replaced with clean figures generated from the same numbers. Code that lost its line breaks in the chat export has been restored to properly formatted Python. Where the original contained a numerical slip, it has been corrected and marked with a note like this one.

Module 1 · Vectors, Matrices, Tensors & Dimensions

Topic: Vectors, matrices, tensors, dimensions · **Output:** hand-worked examples

Let's start Week 4 of your AI Engineering roadmap with four foundational concepts: **vectors, matrices, tensors, and dimensions**.

The focus is on hand-worked examples, so you can calculate everything on paper before seeing how Python represents it.

1. Vectors

A **vector** is an ordered collection of numbers. In machine learning, these numbers often represent the features of one data sample.

For example, imagine a house described by three features:

- Size: 120 square meters
- Bedrooms: 3
- Age: 10 years

We can represent this house as a vector:

$$\mathbf{x} = \begin{bmatrix} 120 \\ 3 \\ 10 \end{bmatrix}$$

This is a vector with 3 elements, also called a **3-dimensional vector**.

Hand-worked example 1: Vector addition

Suppose we have two vectors:

$$\mathbf{a} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}$$

Add corresponding elements:

$$\mathbf{a} + \mathbf{b} = \begin{bmatrix} 2+4 \\ 3+1 \end{bmatrix} = \boxed{\begin{bmatrix} 6 \\ 4 \end{bmatrix}}$$

> **Rule:** vectors must have the same number of elements for ordinary element-wise addition.

Hand-worked example 2: Dot product

Given:

$$\mathbf{a} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

Multiply corresponding elements and add:

$$\mathbf{a} \cdot \mathbf{b} = (2 \times 4) + (3 \times 5)$$
$$= 8 + 15 = \boxed{23}$$

Notice that the result is a **single number**, not another vector.

> AI CONNECTION
> **Why it matters in AI:** a neuron can multiply input features by their weights and add the results to calculate a weighted sum.

2. Matrices

A **matrix** is a rectangular arrangement of numbers organized into rows and columns.

For example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

This matrix has:

- 2 rows
- 3 columns
- 6 elements

Its shape is:

$$\boxed{(2, 3)}$$

The conventional shape notation is **(rows, columns)**.

Hand-worked example 3: Matrix addition

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Add the elements in matching positions:

$$A + B = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix}$$

$$\boxed{\begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}}$$

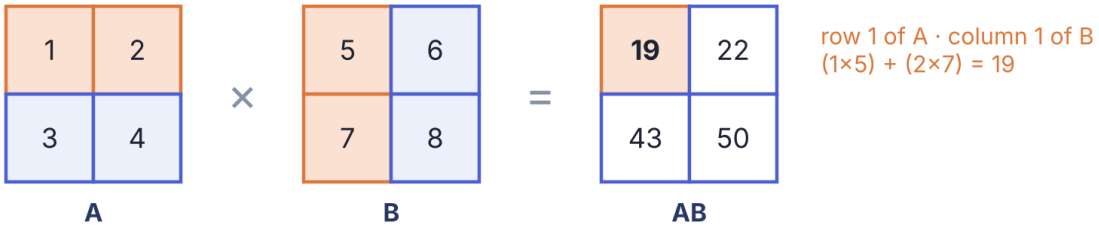
Both matrices must have the same shape for ordinary element-wise addition.

Hand-worked example 4: Matrix multiplication

Given:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

To calculate AB, multiply each **row** of A by each **column** of B.



Each output element is a row of A combined with a column of B.

Top-left element: first row × first column

$$(1 \times 5) + (2 \times 7) = 19$$

Top-right element: first row × second column

$$(1 \times 6) + (2 \times 8) = 22$$

Bottom-left element: second row × first column

$$(3 \times 5) + (4 \times 7) = 43$$

Bottom-right element: second row × second column

$$(3 \times 6) + (4 \times 8) = 50$$

Therefore:

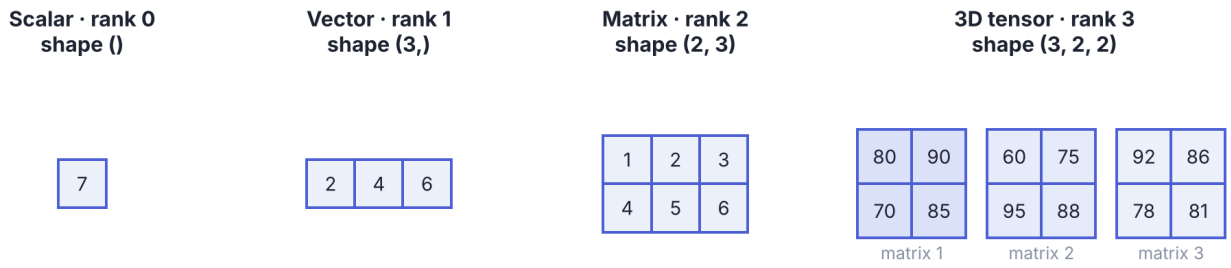
$$\boxed{AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}}$$

> **The important rule:** the number of columns in the first matrix must equal the number of rows in the second matrix.
>
> $$(m, n)(n, p) \longrightarrow (m, p)$$
>
> For our example: $(2, 2)(2, 2) \longrightarrow (2, 2)$

3. Tensors

A **tensor** is a general term for a numerical array that can have any number of axes, or dimensions in the array-structure sense.

Vectors and matrices are tensors, too. The term *tensor* becomes especially useful when working with batches of data, images, and neural networks.



From scalar to 3D tensor: each step adds one axis.

Scalar · 0D tensor. $x = 7$. One number. Shape: `()`.

Vector · 1D tensor. $x = [2, 4, 6]$. One axis containing three elements. Shape: `(3)`, conventionally written in Python as `(3,)`.

Matrix · 2D tensor. $X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$. Two axes: rows and columns. Shape: `(2,3)`.

Now let's work through a 3D tensor by hand.

Hand-worked example 5: A batch of matrices

Suppose you have three students, and each student has two test scores.

$$T = \left[\begin{bmatrix} 80 & 90 \\ 70 & 85 \end{bmatrix}, \begin{bmatrix} 60 & 75 \\ 95 & 88 \end{bmatrix}, \begin{bmatrix} 92 & 86 \\ 78 & 81 \end{bmatrix} \right]$$

We have:

- 3 matrices (one per student group)
- 2 rows in each matrix
- 2 columns in each matrix

Therefore, the shape is:

$$\boxed{(3, 2, 2)}$$

There are $3 \times 2 \times 2 = 12$ numbers in total.

In deep learning, tensors frequently represent **batches of inputs**. For instance, a batch of 32 RGB images of size 224 × 224 might have shape:

$$(32, 224, 224, 3)$$

This convention means: `(batch size, height, width, color channels)`.

> Some frameworks instead use **channels first**, such as `(32, 3, 224, 224)`. Always check the convention used by your model.

4. Dimensions, shape, and rank

This is where terminology can get confusing, because *dimension* has two related meanings.

- **Vector dimension:** the number of components in a vector.
- **Tensor rank:** the number of axes in an array.
- **Shape:** the size along each axis.

Object	Shape	Tensor rank	Number of elements
Scalar `7`	`()`	0	1
Vector `[2,4,6]`	`(3,)`	1	3
Matrix, 2 rows × 3 columns	`(2,3)`	2	6
Tensor of 3 matrices, each 2 × 2	`(3,2,2)`	3	12

For example, the vector $[2, 4, 6]$ has 3 components and tensor rank 1. It is called a **3-dimensional vector** in linear algebra, but a **1D array** in NumPy.

Hand-worked example 6: Read the shape

Consider this tensor:

$$X = \left[\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \begin{bmatrix} 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix}\right]$$

- **Step 1:** Count the matrices: 2.
- **Step 2:** Count the rows in each matrix: 2.
- **Step 3:** Count the columns in each matrix: 3.

So:

$$\boxed{\text{shape}(X) = (2, 2, 3)}$$

Its rank is 3, and it contains $2 \times 2 \times 3 = 12$ elements.

5. Your turn: solve these by hand

Try these without Python first.

Exercise 1 · Vector addition

$$\begin{bmatrix} 3 \\ 5 \end{bmatrix} + \begin{bmatrix} 2 \\ 4 \end{bmatrix} = ?$$

Exercise 2 · Dot product

$$[2, 3] \cdot [4, 6] = ?$$

Exercise 3 · Matrix shape

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix}$$

What is the shape of A?

Exercise 4 · Matrix multiplication

Calculate just the top-left element of AB:

$$A = \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Exercise 5 · Tensor shape and rank

A tensor contains 4 matrices, each with 3 rows and 2 columns. What are its shape, rank, and total number of elements?

YOUR SUBMITTED ANSWERS

1. Vector addition: `[5, 9]`
2. Dot product: `26`
3. Matrix shape: `(4, 3)`
4. Top-left matrix multiplication element: `[[31, 36], [33, 38]]`
5. Tensor: shape = `(4, 3, 2)`, rank = `3`, total number of elements = `24`

6. Review of your answers

Parsa, excellent work! You got 4 out of 5 essentially correct, and the one point to look at is a very useful learning opportunity. Let's check each answer step by step.

Score: 4/5. Exercises 1, 2, 3, and 5 are correct. Exercise 4 needs a closer look.

Exercise 1 · Vector addition CORRECT

Your answer: `[5, 9]`

$$\begin{bmatrix}3\\5\end{bmatrix}+\begin{bmatrix}2\\4\end{bmatrix}=\begin{bmatrix}3+2\\5+4\end{bmatrix}=\boxed{\begin{bmatrix}5\\9\end{bmatrix}}$$

Why it matters: vector addition operates element by element. You did this correctly.

Exercise 2 · Dot product CORRECT

Your answer: `26`

Multiply corresponding elements and sum them:

$$[2,3]\cdot[4,6]=(2\times4)+(3\times6)=8+18=\boxed{26}$$

Key takeaway: the dot product of two vectors produces a scalar (a single number). This operation is fundamental to calculating weighted sums in neural networks.

Exercise 3 · Matrix shape CORRECT

Your answer: `(4, 3)`

Your matrix has four rows and three columns.

$$\boxed{\text{shape}(A)=(4,3)}$$

Remember the order: $\boxed{(\text{rows},\text{columns})}$. A useful habit is to count the rows first, then the entries in each row.

Exercise 4 · Matrix multiplication MORE THAN ASKED

Your answer:

$$\begin{bmatrix}31&36\\33&38\end{bmatrix}$$

The question asked for only the **top-left element** of AB. Your answer is actually the **entire matrix product**, and it is correct!

Let's verify it by hand. Given:

$$A=\begin{bmatrix}2&3\\1&4\end{bmatrix},\quad B=\begin{bmatrix}5&6\\7&8\end{bmatrix}$$

The rule is to take a row from A, take a column from B, multiply matching positions, and sum the results.

For the top-left element, use the first row of A and the first column of B:

$$(2\times5)+(3\times7)=10+21=\boxed{31}$$

Now let's verify your complete answer:

- Top-left: $2(5)+3(7)=31$
- Top-right: $2(6)+3(8)=36$
- Bottom-left: $1(5)+4(7)=33$
- Bottom-right: $1(6)+4(8)=38$

Therefore:

$$\boxed{AB = \begin{bmatrix} 31 & 36 \\ 33 & 38 \end{bmatrix}}$$

Your calculation is completely correct. You simply went beyond what the exercise requested. That's a good sign that you understand the operation.

Exercise 5 · Tensor shape, rank, and element count CORRECT

Your answer: shape `(4, 3, 2)`, rank `3`, total elements `24`.

Step 1 · Shape. There are four matrices, each with three rows and two columns: $\boxed{(4, 3, 2)}$

Step 2 · Rank. The shape has three axes: matrix index, row, and column. Therefore, the tensor rank is 3.

Step 3 · Number of elements. Multiply the sizes of all axes: $4 \times 3 \times 2 = \boxed{24}$

Everything is correct.

What you should remember from this lesson

Concept	Essential rule
Vector addition	Add corresponding elements
Dot product	Multiply corresponding elements, then sum
Matrix shape	`(rows, columns)`
Matrix multiplication	Each output element is a row-column dot product
Tensor shape	Lists the size of every axis
Tensor rank	Number of axes
Number of elements	Multiply all shape values

> **A particularly important distinction for your AI engineering journey:** tensor rank is not the same as the number of elements or the dimension of a vector. A vector with 100 components is still a rank-1 tensor.

Next step suggested in the session: vector norms and magnitude, which cover how to calculate the length of a vector, why normalization matters for embeddings, and how cosine similarity uses vector geometry to compare meaning in AI systems.

Module 2 · Dot Product & Matrix Multiplication with NumPy

Topic: Dot product and matrix multiplication · **Output:** NumPy implementation

Let's move from hand calculations to Python. The goal is to understand how to implement both operations with NumPy and, more importantly, **when to use each one** in machine learning.

We'll use `np.dot()`, `@`, and `np.matmul()` and compare their behavior.

1. Dot product of two vectors

Recall the hand-worked example:

$$[2, 3] \cdot [4, 6] = (2 \times 4) + (3 \times 6) = 26$$

In NumPy:

```
import numpy as np

a = np.array([2, 3])
b = np.array([4, 6])

result = np.dot(a, b)
print(result)
# 26
```

How does NumPy calculate this?

Vector A	Vector B	Products	Sum
`[2, 3]`	`[4, 6]`	$2 \times 4 = 8$ and $3 \times 6 = 18$	$8 + 18 = \mathbf{26}$

The two vectors must have the same length. Their dot product returns a single scalar.

You can also use the `@` operator:

```
result = a @ b
print(result)  # 26
```

For two 1D NumPy arrays, `np.dot(a, b)` and `a @ b` produce the same result.

2. Matrix multiplication

Now suppose we have two matrices:

$$A = \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

The result is:

$$AB = \begin{bmatrix} 31 & 36 \\ 33 & 38 \end{bmatrix}$$

Let's implement it.

```
import numpy as np

A = np.array([
    [2, 3],
    [1, 4]
])

B = np.array([
    [5, 6],
    [7, 8]
])

result = A @ B
print(result)
```

Output:

```
[[31 36]
 [33 38]]
```

Three ways to do matrix multiplication

```
result1 = A @ B
result2 = np.matmul(A, B)
result3 = np.dot(A, B)

print(np.array_equal(result1, result2))  # True
print(np.array_equal(result1, result3))  # True
```

For these two 2D matrices, all three produce the same answer.

Operation	Recommended use
`np.dot(a, b)`	Dot product of two vectors; also supports other dimensional combinations
`A @ B`	Matrix multiplication using readable, concise syntax
`np.matmul(A, B)`	Explicit matrix multiplication, including batched operations

Recommendation: learn `@` as your default syntax for matrix multiplication. Use `np.dot()` when you specifically mean a vector dot product, although NumPy supports other uses for it too.

3. The most important rule: matrix shapes

Not every pair of matrices can be multiplied. If

$$A : (m, n), \qquad B : (n, p)$$

then

$$AB : (m, p)$$

The **inner dimensions must match.** For example:

$$(2, 3) \times (3, 4) \to (2, 4)$$

This multiplication is valid because both inner dimensions are 3. Let's verify it with NumPy.

```
A = np.array([
    [1, 2, 3],
    [4, 5, 6]
])  # shape (2, 3)

B = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]
])  # shape (3, 4)

C = A @ B

print(A.shape)  # (2, 3)
print(B.shape)  # (3, 4)
print(C.shape)  # (2, 4)
print(C)
```

Output:

```
(2, 3)
(3, 4)
(2, 4)

[[ 38  44  50  56]
 [ 83  98 113 128]]
```

Why does the output have shape `(2, 4)`? Each of the two rows in `A` produces four output values, one for each column in `B`.

4. Matrix multiplication is not element-wise multiplication

This distinction is essential.

```
A = np.array([
    [1, 2],
    [3, 4]
])

B = np.array([
    [5, 6],
    [7, 8]
])

print(A @ B)
print(A * B)
```

Output:

```
# A @ B: matrix multiplication
[[19 22]
 [43 50]]

# A * B: element-wise multiplication
[[ 5 12]
 [21 32]]
```

For element-wise multiplication, NumPy multiplies numbers at corresponding positions:

$$A * B = \begin{bmatrix} 1(5) & 2(6) \\ 3(7) & 4(8) \end{bmatrix}$$

For matrix multiplication, it calculates row-column dot products instead.

A common mistake is to use `*` when the algorithm requires `@`.

5. Why this matters in AI engineering

Imagine a batch of 100 samples, each with 3 features, and a linear model with 2 outputs.

Your input matrix has shape $X : (100, 3)$ and your weight matrix has shape $W : (3, 2)$. The predictions are calculated by:

$$\hat{Y} = XW$$

The result has shape:

$$(100, 3) \times (3, 2) = (100, 2)$$

In NumPy:

```
X = np.random.rand(100, 3)
W = np.random.rand(3, 2)

Y_pred = X @ W
print(Y_pred.shape)
# (100, 2)
```

This is a simplified **linear layer**: 100 samples enter, and each produces 2 outputs. A bias term can be added afterward.

AI CONNECTION

The same underlying operation appears in neural networks, where input features are multiplied by learned weights to produce activations.

6. Your turn: NumPy practice

Exercise 1 · Vector dot product

Given `a = np.array([3, 2, 1])` and `b = np.array([4, 5, 6])`, write the NumPy expression to calculate their dot product and give the result.

Exercise 2 · Matrix multiplication

Given `A = np.array([[1, 2], [3, 4]])` and `B = np.array([[2, 0], [1, 2]])`, calculate `A @ B`.

Exercise 3 · Shape compatibility

Can a matrix with shape `(4, 3)` be multiplied by one with shape `(2, 5)` using `@`? Explain why or why not.

Exercise 4 · Understand the output shape

If `X.shape == (64, 10)` and `W.shape == (10, 4)`, what will `(X @ W).shape` be, and what might the dimensions represent in an ML model?

YOUR SUBMITTED ANSWERS

1. `np.dot(a, b)` → output: `28`
2. `[[4, 4], [10, 8]]`
3. No, it cannot. In `(a, b) @ (c, d)`, `b` must be equal to `c`.
4. `(64, 4)`

7. Review of your answers

Excellent work, Parsa! You got **4 out of 4** correct. More importantly, your answer to Exercise 3 shows that you understand the fundamental rule behind matrix multiplication. Let's verify each answer and strengthen your intuition.

Score: 4/4. All answers correct.

Exercise 1 · Vector dot product CORRECT

Your code: `np.dot(a, b)`, given `a = np.array([3, 2, 1])` and `b = np.array([4, 5, 6])`.

Multiply corresponding elements, then add:

$$a \cdot b = (3 \times 4) + (2 \times 5) + (1 \times 6)$$
$$= 12 + 10 + 6 = \boxed{28}$$

AI CONNECTION

AI connection: a dot product measures how two vectors align and is used to calculate weighted sums in neural networks. It is also the basis for cosine similarity, which compares vector directions after normalization.

Exercise 2 · Matrix multiplication CORRECT

Your answer:

$$\begin{bmatrix} 4 & 4 \\ 10 & 8 \end{bmatrix}$$

Given:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$$

Let's calculate each element.

- Top-left: $1(2) + 2(1) = 4$
- Top-right: $1(0) + 2(2) = 4$
- Bottom-left: $3(2) + 4(1) = 10$
- Bottom-right: $3(0) + 4(2) = 8$

Therefore:

$$\boxed{AB = \begin{bmatrix} 4 & 4 \\ 10 & 8 \end{bmatrix}}$$

NumPy implementation:

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[2, 0], [1, 2]])

print(A @ B)
```

Output:

```
[[ 4  4]
 [10  8]]
```

Exercise 3 · Shape compatibility CORRECT

Your explanation: in `(a, b) @ (c, d)`, `b` must equal `c`. **Exactly right!**

The general rule is:

$$(m, n) \mathbin{@} (n, p) \to (m, p)$$

For your example, $(4, 3) \mathbin{@} (2, 5)$: the inner dimensions are 3 and 2. Since $3 \neq 2$, the multiplication is invalid.

First matrix	Second matrix	Inner dimensions	Result
`(4, 3)`: 4 rows, 3 columns	`(2, 5)`: 2 rows, 5 columns	3 vs 2	**Invalid: 3 ≠ 2**

You can verify this in Python:

```
A = np.zeros((4, 3))
B = np.zeros((2, 5))

C = A @ B  # ValueError: incompatible dimensions
```

NumPy raises an error because the inner dimensions do not match.

A useful refinement: matrix multiplication is not necessarily commutative. Even if both AB and BA are valid, they generally produce different results. In some cases, only one order is valid.

Exercise 4 · Output shape in machine learning CORRECT

Your answer: `(64, 4)`.

Given X with shape `(64, 10)` and W with shape `(10, 4)`:

$$(64, 10) \mathbin{@} (10, 4) = \boxed{(64, 4)}$$

The inner dimensions match, and the output takes its outer dimensions from the first and second matrices. In a simple neural network layer:

- `64` = number of samples in the batch.
- `10` = input features per sample.
- `4` = output values produced per sample.

So each of the 64 samples gets 4 outputs.

```
X = np.random.rand(64, 10)
W = np.random.rand(10, 4)

Y = X @ W
print(Y.shape)  # (64, 4)
```

This is the core operation behind a **fully connected neural network layer**, before adding bias and applying an activation function.

Your progress: what you've mastered

Skill	What it means	Status
Vector dot product	Multiply matching elements and sum	Mastered
Matrix multiplication	Calculate each output using row-column products	Mastered
Shape compatibility	Match the inner dimensions	Mastered
ML batch dimensions	Interpret input and output shapes	Mastered

Next topic suggested in the session: vector norms (magnitude) and normalization: calculating the L1 and L2 norms by hand and with NumPy, then connecting normalization to cosine similarity and embeddings.

Module 3 · Probability, Conditional Probability & Bayes' Theorem

Topic: Probability, conditional probability, Bayes · **Output:** Bayes exercises

This module focuses on **Bayes exercises**, with practical examples and increasing difficulty. Try solving each one by hand before checking the answers.

Quick reference

Probability of an event:

$$P(A) = \frac{\text{favorable outcomes}}{\text{total equally likely outcomes}}$$

Conditional probability: the probability of A given that B has occurred:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Bayes' theorem lets us reverse a conditional probability:

$$P(A \mid B) = \frac{P(B \mid A)\,P(A)}{P(B)}$$

A worked mini-example

Suppose $P(A) = 0.10$, $P(B \mid A) = 0.90$ and $P(B \mid \neg A) = 0.10$.

Quantity	Calculation	Value
$P(B \cap A)$	$P(B \mid A)P(A) = 0.90(0.10)$	0.09
$P(B \cap \neg A)$	$P(B \mid \neg A)P(\neg A) = 0.10(0.90)$	0.09
$P(B)$	$0.09 + 0.09$	0.18
$P(A \mid B)$	$\frac{0.90(0.10)}{0.18}$	$\mathbf{0.50}$

> **The key idea:** the probability of evidence given a hypothesis is *not* the same as the probability of the hypothesis given the evidence. Above, $P(B \mid A) = 0.90$ but $P(A \mid B) = 0.50$.

Exercises

Exercise 1 · Basic conditional probability

A box contains 6 red balls and 4 blue balls. You randomly pick one ball, and you are told it is **not blue**. What is the probability that the ball is red? Write your answer as a fraction and a percentage.

Exercise 2 · Medical testing

A disease affects 1% of a population. A test has:

- **Sensitivity: 95%**: it correctly tests positive for 95% of people who have the disease.
- **False-positive rate: 5%**: it tests positive for 5% of people who do not have the disease.

A randomly selected person tests positive. What is the probability that this person actually has the disease?

Calculate these first: (1) $P(D)$, the probability of disease; (2) $P(+ \mid D)$, the probability of a positive test given disease; (3) $P(+ \mid \neg D)$, the probability of a positive test without disease; (4) apply Bayes' theorem to find $P(D \mid +)$.

Exercise 3 · Spam detection

- 30% of incoming emails are spam.
- The filter flags 90% of spam emails.
- The filter incorrectly flags 10% of legitimate emails.

An email is flagged. What is the probability that it is actually spam? Define your events, calculate the overall probability of a flagged email, and then apply Bayes' theorem.

Exercise 4 · Machine learning classification

A machine learning model classifies transactions as fraudulent or legitimate.

- 2% of transactions are fraudulent.
- The model flags 98% of fraudulent transactions.
- The model also flags 3% of legitimate transactions.

A transaction is flagged as fraudulent. What is the probability that it really is fraudulent? Explain why the model's 98% detection rate does not mean that 98% of flagged transactions are fraudulent.

Exercise 5 · Challenge: Two factories

- Factory A produces 70% of all products, and 2% of its products are defective.
- Factory B produces 30% of all products, and 5% of its products are defective.

You randomly select a product and discover that it is defective. What is the probability that it came from Factory B? You'll need to calculate the total probability of selecting a defective product before applying Bayes' theorem.

Exercises 2 to 5 are especially useful for understanding how Bayes' theorem applies to real-world AI classification.

YOUR SUBMITTED ANSWERS

1. 1, i.e. 100%
2. $P(D) = 0.01$, $P(+ \mid D) = 0.95$, $P(+ \mid \neg D) = 0.05$, $P(D \mid +) = \dfrac{P(+ \mid D)\,P(D)}{P(+)} = \dfrac{0.0095}{0.059} \approx 0.161$
3. $\dfrac{0.9 \times 0.3}{0.9 \times 0.3 + 0.1 \times 0.7} = 0.79$; overall flagged: $0.9 \times 0.3 + 0.1 \times 0.7 = 0.34$
4. $\dfrac{0.98 \times 0.02}{0.98 \times 0.02 + 0.03 \times 0.98} = 0.4$, because 0.98 is only the share of fraudulent transactions detected as fraudulent, but legitimate transactions get detected as fraudulent too.
5. $\dfrac{0.3 \times 0.05}{0.02 \times 0.7 + 0.05 \times 0.3} = 0.5172$

Review of your answers

You did really well: **all five exercises are mathematically correct.** More importantly, in Exercises 2 to 5, you correctly used Bayes' theorem to account for both true positives and false positives. That's the central idea to understand.

Score: 5/5. All answers correct.

Exercise 1 · Conditional probability CORRECT

Your answer: 1, or 100%.

The box contains 6 red balls and 4 blue balls. We know the selected ball is not blue. The conditional probability formula is:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Let A = selecting a red ball, and B = selecting a ball that is not blue. Since the box contains only red and blue balls, knowing the ball is not blue means it must be red.

$$P(\text{red} \mid \text{not blue}) = \frac{6/10}{6/10} = 1$$

Intuition: the original probability of red was 60%, but once we know the ball isn't blue, the uncertainty disappears. The probability becomes 100%.

Exercise 2 · Medical test and Bayes' theorem CORRECT

Your answer: $P(D \mid +) \approx 0.161$, or 16.1%. You correctly identified all the probabilities:

- $P(D) = 0.01$: 1% of people have the disease.
- $P(+ \mid D) = 0.95$: the test is positive for 95% of people who have it.
- $P(+ \mid \neg D) = 0.05$: 5% of people without the disease get a false positive.

First, calculate the overall probability of a positive test. This uses the **law of total probability**:

$$P(+) = P(+ \mid D)P(D) + P(+ \mid \neg D)P(\neg D)$$

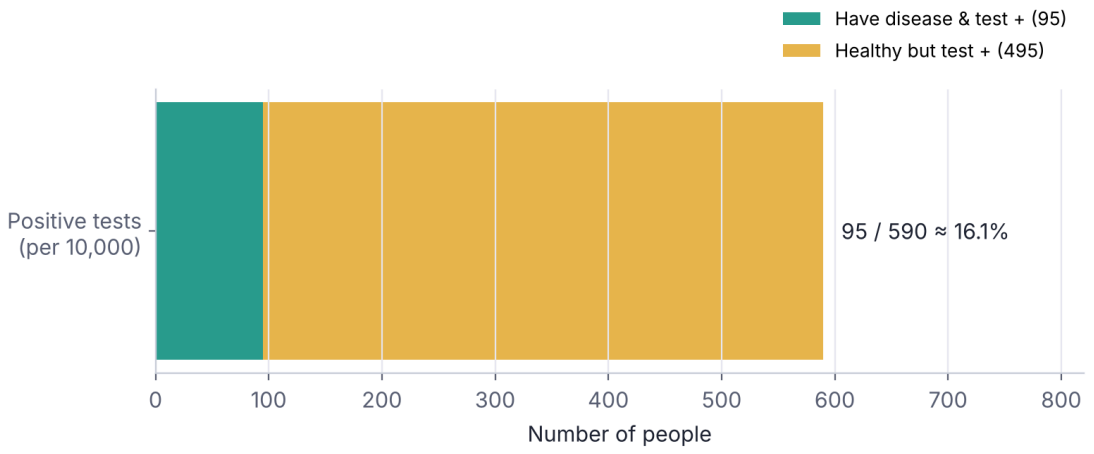
Since $P(\neg D) = 1 - 0.01 = 0.99$:

$$P(+) = 0.95(0.01) + 0.05(0.99) = 0.0095 + 0.0495 = 0.059$$

Now apply Bayes' theorem:

$$P(D \mid +) = \frac{P(+ \mid D)P(D)}{P(+)} = \frac{0.95(0.01)}{0.059} = \frac{0.0095}{0.059} \approx 0.161$$

Final answer: 16.1%.



Out of 10,000 people: 95 true positives against 495 false positives.

The surprising part is that a test with 95% sensitivity can still yield only a 16.1% probability of disease after a positive result. That's because the disease is rare, so false positives among the much larger healthy population add up.

Exercise 3 · Spam detection CORRECT

Your answer: 0.79, or approximately 79%. You wrote:

$$P(S \mid F) = \frac{0.9(0.3)}{0.9(0.3) + 0.1(0.7)}$$

where S means spam and F means flagged. Let's break it down.

Step 1 · True positives. 30% of emails are spam, and the filter flags 90% of spam:

$$P(F \mid S)P(S) = 0.9(0.3) = 0.27$$

So 27% of all emails are spam and get flagged.

Step 2 · False positives. 70% of emails are legitimate, and 10% of legitimate emails are incorrectly flagged:

$$P(F \mid \neg S)P(\neg S) = 0.1(0.7) = 0.07$$

So 7% of all emails are legitimate but get flagged.

Step 3 · All flagged emails.

$$P(F) = 0.27 + 0.07 = 0.34$$

You correctly calculated this as 34%.

Step 4 · Probability that a flagged email is spam.

$$P(S \mid F) = \frac{0.27}{0.34} \approx 0.7941$$

Final answer: 79.41%. Your reasoning is correct. The only improvement is to keep a few more decimal places until the final step.

Exercise 4 · Fraud detection CORRECT

Your answer: 0.4, or 40%. You wrote:

$$\frac{0.98(0.02)}{0.98(0.02) + 0.03(0.98)} = 0.4$$

Let's interpret each term:

Term	Meaning	Calculation
0.02	Fraction of transactions that are fraudulent	2%
0.98	Detection rate for fraudulent transactions	98%
$0.98(0.02)$	All transactions that are fraudulent and flagged	0.0196
$0.03(0.98)$	All transactions that are legitimate but flagged	0.0294

The total fraction of transactions flagged is:

$$P(F) = 0.0196 + 0.0294 = 0.049$$

Then:

$$P(\text{fraud} \mid F) = \frac{0.0196}{0.049} = 0.40$$

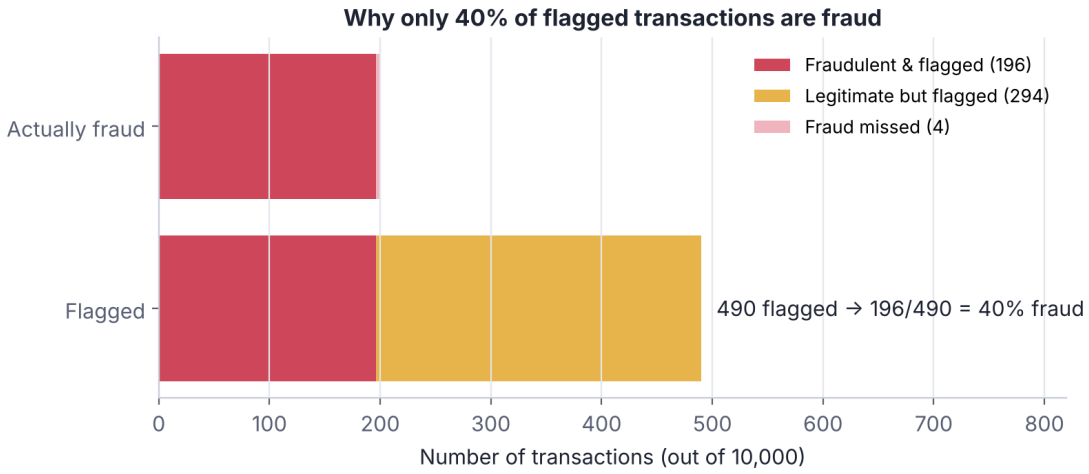
Final answer: 40%.

Your explanation was also correct. One small terminology improvement: 98% is the **true-positive rate** (or **sensitivity**), not the number of fraudulent transactions detected. The crucial distinction is:

- $P(\text{flagged} \mid \text{fraud}) = 98\%$
- $P(\text{fraud} \mid \text{flagged}) = 40\%$

These are different conditional probabilities.

Why is the result only 40%? Imagine 10,000 transactions: **196** fraudulent transactions are correctly flagged, and **294** legitimate transactions are incorrectly flagged.



Of the 490 flagged transactions, only 196 are fraudulent.

$$\frac{196}{490} = 40\%$$

Even a good detector can produce many false alarms when fraud itself is rare.

Exercise 5 · Factory defect rates CORRECT

Your answer: 0.5172, or approximately 51.72%. You correctly calculated the probability that a defective item came from Factory B.

Given: Factory A produces 70% of all items, with a 2% defect rate; Factory B produces 30% of all items, with a 5% defect rate. We want $P(B \mid D)$: the probability an item came from Factory B, given that it is defective.

Step 1 · Defective items from Factory B.

$$P(D \mid B)P(B) = 0.05(0.30) = 0.015$$

So 1.5% of all items are both from B and defective.

Step 2 · Defective items from Factory A.

$$P(D \mid A)P(A) = 0.02(0.70) = 0.014$$

So 1.4% of all items are both from A and defective.

Step 3 · Total defect probability.

$$P(D) = 0.015 + 0.014 = 0.029$$

Therefore, 2.9% of all items are defective.

Step 4 · Apply Bayes' theorem.

$$P(B \mid D) = \frac{P(D \mid B)P(B)}{P(D)} = \frac{0.05(0.30)}{0.029} = \frac{0.015}{0.029} \approx 0.5172$$

Final answer: 51.72%. Although Factory B produces only 30% of all items, it accounts for approximately 51.72% of defective items because its defect rate is higher.

Your Week 4 takeaway

You got all five exercises correct. The most important skill you're developing is distinguishing the probability of evidence given a hypothesis from the probability of the hypothesis given evidence.

Concept	Formula	What it answers
Conditional probability	$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$	Probability of A when B is known
Total probability	$P(B) = \sum_i P(B \mid A_i)P(A_i)$	Overall probability of evidence from all possible cases
Bayes' theorem	$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$	Probability of a hypothesis after observing evidence

Assessment: your formula setup and interpretation are strong. Keep practicing the habit of identifying the **prior**, the **likelihood**, and the overall **evidence probability** before calculating the **posterior**. That habit will be useful later in machine learning, especially for Naive Bayes classification and reasoning about model predictions.

Module 4 · Statistics Notebook: Mean, Variance, Distributions & Correlation

Topic: Mean, variance, distributions, correlation · **Output:** stats notebook

We'll build this like a practical Jupyter notebook for your AI Engineering path. The goal is to understand the statistics, calculate them by hand, and then implement them with NumPy and pandas.

By the end, you should understand:

1. **Mean:** the center or average of your data.
2. **Variance and standard deviation:** how spread out the data is.
3. **Distributions:** how values are spread and how frequently they occur.
4. **Correlation:** how two variables move together.
5. **Python implementation:** calculate and interpret these statistics using real code.

Notebook 1 · Mean, variance, and standard deviation

Cell 1 · Create a dataset

Imagine these are the test scores of five students:

```
import numpy as np
import pandas as pd

scores = np.array([2, 4, 4, 4, 6])
```

We use a small dataset so you can verify the math manually.

Cell 2 · Mean

The **mean** is the sum of all values divided by the number of values.

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i$$

For our data:

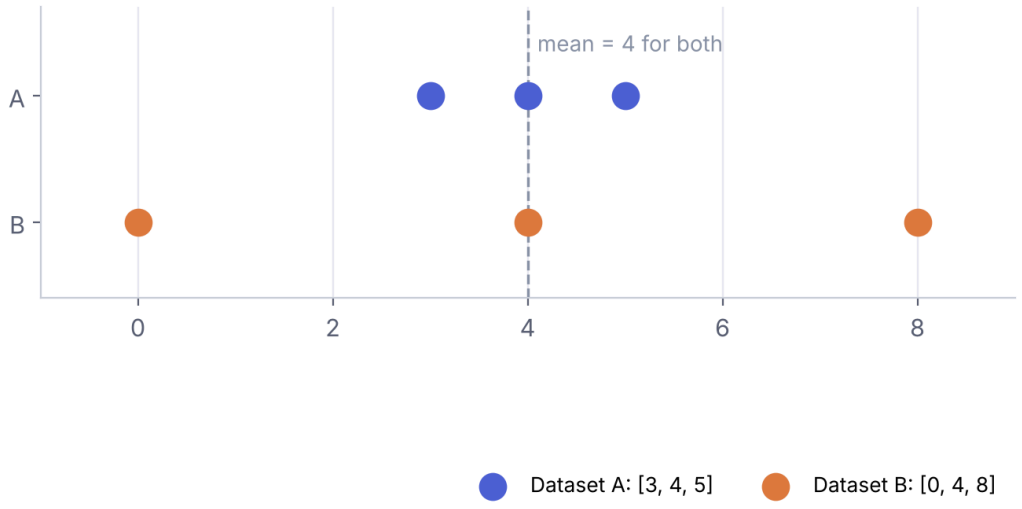
$$\mu = \frac{2+4+4+4+6}{5} = \frac{20}{5} = 4$$

In Python:

```
np.mean(scores)
# 4.0
```

The mean gives us a measure of the dataset's center. But it doesn't tell us whether the values are tightly grouped or widely spread. Consider these two datasets:

- A: `[3, 4, 5]`
- B: `[0, 4, 8]`



Same mean, different spread.

Both have a mean of 4, but Dataset B is more spread out. That's where variance comes in.

Cell 3 · Variance: measure of spread

Variance measures the average squared distance from the mean. For the scores `[2, 4, 4, 4, 6]`, the mean is 4.

x_i	Deviation $x_i - \mu$	Squared deviation
2	−2	4
4	0	0
4	0	0
4	0	0
6	2	4
Total	**0**	**8**

For a **population**, divide the sum of squared deviations by N:

$$\sigma^2 = \frac{\sum_{i=1}^{N}(x_i - \mu)^2}{N}$$

So:

$$\sigma^2 = \frac{8}{5} = 1.6$$

In NumPy:

```
np.var(scores)
# 1.6
```

An equivalent shortcut formula: $\mathrm{Var}(X) = E(X^2) - [E(X)]^2$. For our scores, $E(X^2) = \frac{4+16+16+16+36}{5} = 17.6$ and $[E(X)]^2 = 16$, so $\mathrm{Var}(X) = 17.6 - 16 = 1.6$, the same answer.

Why square the deviations?

- Negative and positive deviations would cancel if we simply added them.
- Squaring makes every contribution nonnegative.
- Larger deviations contribute more heavily to the result.

Cell 4 · Standard deviation

Variance is measured in squared units. **Standard deviation** brings the result back to the original units by taking the square root.

$$\sigma = \sqrt{\sigma^2} = \sqrt{1.6} \approx 1.265$$

```
np.std(scores)
# 1.2649...
```

If these values are exam scores, the standard deviation is about 1.265 score points.

One important distinction: `np.var()` and `np.std()` default to `ddof=0`, which calculates **population** statistics. For a **sample** used to estimate a larger population, we commonly use `ddof=1`.

```
np.var(scores, ddof=1)  # sample variance: 2.0
np.std(scores, ddof=1)  # sample standard deviation: about 1.414
```

Why does sample variance divide by $N-1$ rather than N? Because estimating the mean from the same sample tends to make the deviations look smaller; dividing by $N-1$ corrects that bias under standard sampling assumptions.

Notebook 2 · Distributions

A **distribution** describes what values a variable can take and how likely or frequent those values are.

For example, if we record the scores of 100 students, the distribution tells us whether most students scored around 50, whether scores are concentrated near 90, or whether there are two distinct groups.

Cell 5 · Explore a distribution

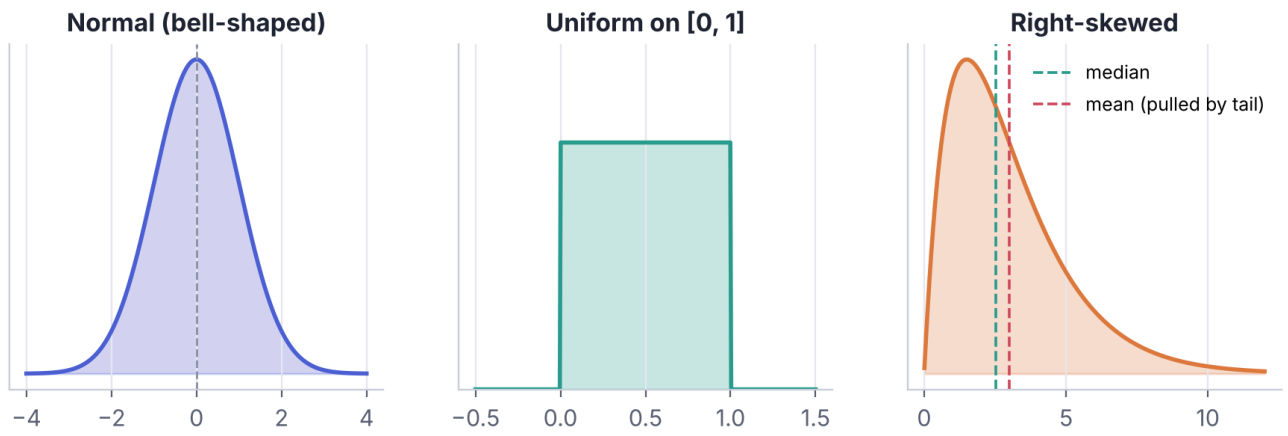
```
scores = np.array([2, 4, 4, 4, 6])

# Frequency of each value
values, counts = np.unique(scores, return_counts=True)

print(values)   # [2 4 6]
print(counts)   # [1 3 1]
```

Here, 4 appears three times, while 2 and 6 each appear once.

Some common distribution types you'll encounter in machine learning:



Three common distribution shapes.

1. **Normal distribution.** A symmetric, bell-shaped distribution where values cluster around the mean. Examples include some measurement errors and approximately normal biological measurements. Common in statistics and many ML assumptions.
2. **Uniform distribution.** Values in a given range have equal probability density in the continuous case. For example, a random number drawn uniformly between 0 and 1. Useful for random sampling and initialization.
3. **Skewed distribution.** Values cluster more on one side, with a longer tail on the other. Income and some transaction amounts are often right-skewed. The mean can be pulled toward the long tail.

Cell 6 · Mean versus median

The **median** is the middle value after sorting the data. It can better represent the center when extreme values are present.

```
incomes = np.array([20, 22, 23, 25, 110])

print(np.mean(incomes))    # 40.0
print(np.median(incomes))  # 23.0
```

The value 110 pulls the mean upward, while the median remains 23.

AI CONNECTION

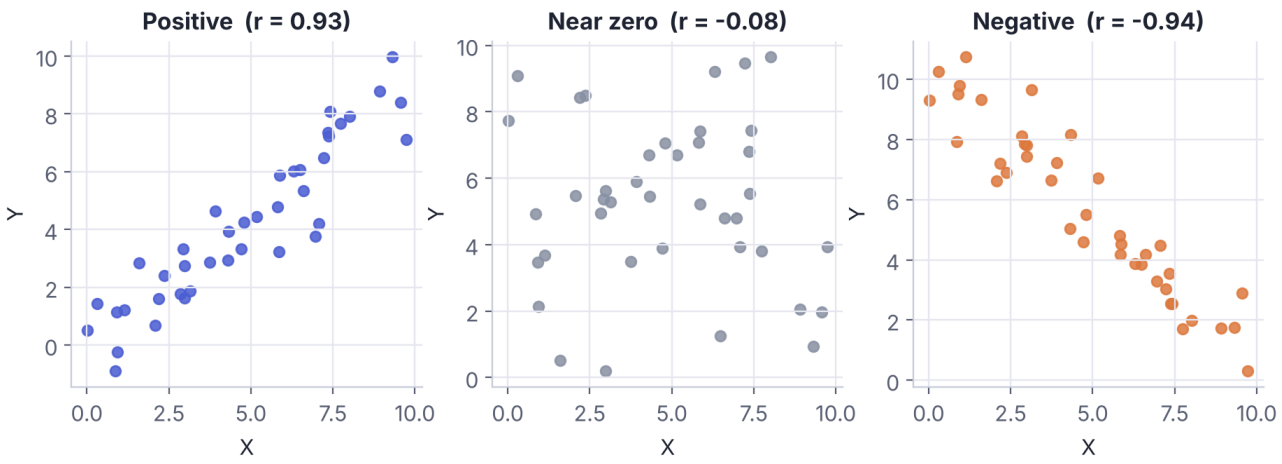
In real datasets, inspecting a distribution helps you spot outliers, skewness, unusual values, and potential data-quality issues **before training a model**.

Notebook 3 · Correlation

Correlation measures the direction and strength of a relationship between two variables. **Pearson correlation** measures their *linear* relationship. For Pearson's correlation coefficient r:

$$-1 \leq r \leq 1$$

- $r \approx 1$: strong positive linear relationship.
- $r \approx -1$: strong negative linear relationship.
- $r \approx 0$: little or no linear relationship.



Positive, near-zero, and negative correlation.

Cell 7 · Calculate correlation with NumPy

Suppose we record hours studied and exam scores:

```
hours = np.array([1, 2, 3, 4, 5])
scores = np.array([2, 4, 5, 4, 5])

correlation_matrix = np.corrcoef(hours, scores)
print(correlation_matrix)
```

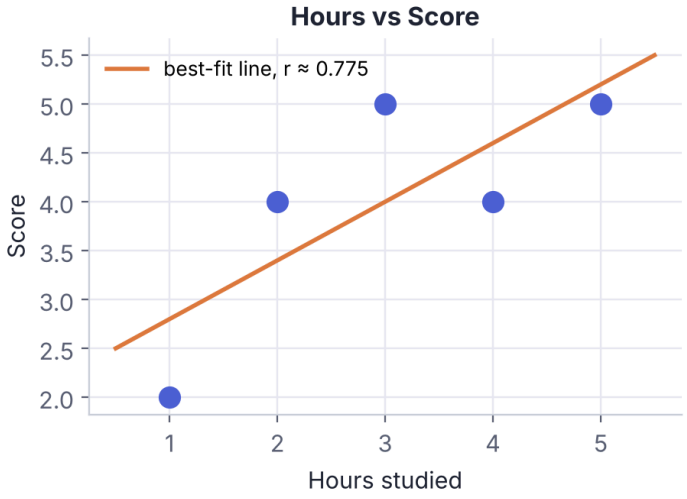
The output is a 2 × 2 matrix:

```
[[1.         0.7746...]
 [0.7746...  1.        ]]
```

The diagonal is 1 because each variable is perfectly correlated with itself. The off-diagonal entries are the correlation between `hours` and `scores`.

Here, the correlation is approximately **0.775**, indicating a fairly strong positive linear relationship in this small dataset.

> **Correction:** the original chat stated this value as 0.832. Running the code gives $r = 0.7746$, so the correct value is used here.



The positive trend between hours and score.

Cell 8 · Calculate correlation with pandas

```
df = pd.DataFrame({
    "hours": [1, 2, 3, 4, 5],
    "scores": [2, 4, 5, 4, 5]
})

df.corr()
```

For multiple numeric columns, `df.corr()` returns a correlation matrix, which is useful during exploratory data analysis.

What does correlation actually calculate?

Pearson correlation standardizes the covariance between two variables:

$$r = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

The **covariance** tells us whether the variables tend to be above their means together or on opposite sides of their means. Dividing by their standard deviations makes the correlation unitless and bounded between −1 and 1.

> **Important: correlation does not prove causation.** For example, ice cream sales and swimming incidents might rise together because both are influenced by hot weather.

Notebook 4 · Put it all together

Cell 9 · A small statistical analysis

Let's create a DataFrame and calculate several statistics at once.

```
df = pd.DataFrame({
    "hours": [1, 2, 3, 4, 5],
    "scores": [2, 4, 5, 4, 5]
})

# Summary statistics
print(df.describe())

# Variance and standard deviation
print(df.var())   # sample variance by default
print(df.std())   # sample standard deviation by default

# Correlation
print(df.corr())
```

> Notice an important detail: **pandas uses sample variance and sample standard deviation by default**, whereas NumPy's `np.var()` and `np.std()` use population formulas by default. This is why the results may differ unless you set `ddof` consistently.

Cell 10 · Visualize the data

```
import matplotlib.pyplot as plt

# Distribution of scores
plt.hist(df["scores"], bins=5)
plt.xlabel("Score")
plt.ylabel("Frequency")
plt.title("Distribution of Scores")
plt.show()

# Relationship between hours and scores
plt.scatter(df["hours"], df["scores"])
plt.xlabel("Hours studied")
plt.ylabel("Score")
plt.title("Hours vs Score")
plt.show()
```

Use a **histogram** to inspect the distribution of a single variable, and a **scatter plot** to inspect the relationship between two variables.

Quick reference

Statistic	What it tells you	Python
Mean	Average / center	`np.mean(x)`
Median	Middle value	`np.median(x)`
Population variance	Spread, squared units	`np.var(x)`
Standard deviation	Spread, original units	`np.std(x)`
Distribution	How values are spread	`plt.hist(x)`
Pearson correlation	Direction and strength of a linear relationship	`np.corrcoef(x, y)[0, 1]`

The goal isn't just to memorize functions; it's to be able to look at a dataset and explain what its statistics tell you about the data.

Practice exercises

Try solving these without looking up the answers. You can use Python to verify your calculations after doing them by hand.

Exercise 1 · Mean

Given `x = [2, 4, 6, 8]`, calculate the mean by hand and then using NumPy.

Exercise 2 · Variance

For the same dataset, calculate the population variance by hand. Show the mean, deviations, squared deviations, and final variance.

Exercise 3 · Outliers and distributions

For `x = [10, 10, 10, 10, 60]`, predict which is larger: the mean or the median. Explain why.

Exercise 4 · Correlation

Consider `a = [1, 2, 3, 4]`, `b = [2, 4, 6, 8]`, `c = [8, 6, 4, 2]`. Without running the code, predict the Pearson correlation of `a` with `b`, and `a` with `c`. Explain the direction of each relationship.

Exercise 5 · NumPy and pandas

Write Python code to calculate the mean, population variance, standard deviation, and correlation for two arrays of your choice. Explain the difference between `np.var(x)` and `pd.Series(x).var()`.

YOUR SUBMITTED ANSWERS

1. `5`, `np.mean(x)`
2. Mean `5`; deviations `[3, 1, -1, -3]`; squared `[9, 1, 1, 9]`; variance $20/4 = 5$
3. The mean is larger because the distribution is not normal and is concentrated toward 10, so the mean is 20 and the median is 10.
4. `1`, `-1`
5. `np.mean(x)`, `np.var(x)`, `np.std(x)`, `np.corr(x, y)`. Pandas uses sample statistics but NumPy uses population statistics.

Review of your answers

You've got the main ideas right. This scores **4/5 conceptually**, with a few details to fix in the variance working and Python syntax. Let's go through each exercise.

Score: 4/5 conceptually. Every numeric answer is right; the deductions are for precise wording and one Python function name.

Exercise 1 · Mean CORRECT

Your answer: `5`. Dataset: `x = [2, 4, 6, 8]`. The mean is the sum divided by the number of values:

$$\bar{x} = \frac{2+4+6+8}{4} = \frac{20}{4} = 5$$

Your NumPy code is correct, assuming `x` is defined:

```
np.mean(x)  # 5.0
```

Exercise 2 · Population variance CORRECT, MINOR NOTE

Your answer: 5. The final answer is correct, but your working has a small inconsistency.

Value x_i	Deviation $x_i - \mu$	Squared deviation
2	−3	9
4	−1	1
6	1	1
8	3	9
Total	**0**	**20**

The mean is 5. Population variance is the average of the squared deviations:

$$\sigma^2 = \frac{\sum_{i=1}^{N}(x_i - \mu)^2}{N} = \frac{20}{4} = 5$$

So your final answer is correct! The correction is just that the deviation list should be `[-3, -1, 1, 3]`, not `[3, 1, -1, -3]`. You listed the signs in reverse order, but squaring makes the final variance unchanged.

> **Editor's note:** the original review marked this exercise with a ❌ while also saying the answer was correct. It is shown here as correct, with the sign-order note.

In Python:

```
np.var(x)  # 5.0, population variance
```

And the standard deviation would be $\sigma = \sqrt{5} \approx 2.236$.

Exercise 3 · Mean versus median CORRECT

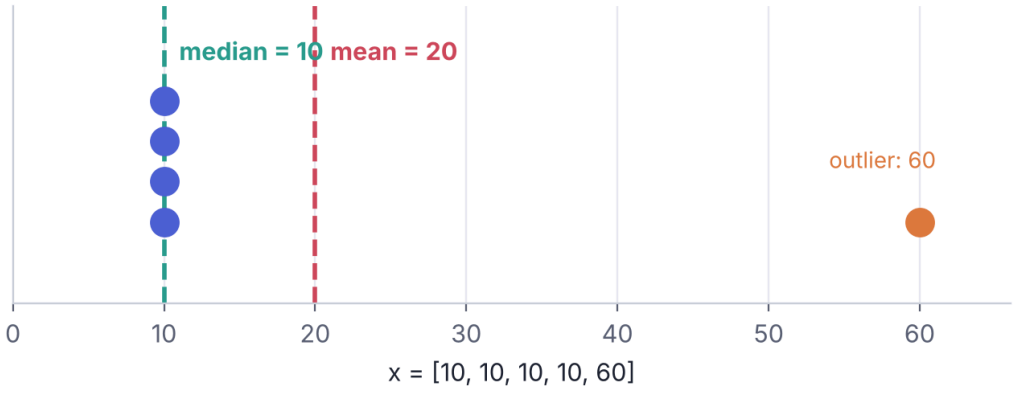
Your answer: mean = 20, median = 10. Dataset: `x = [10, 10, 10, 10, 60]`.

Step 1 · Calculate the mean.

$$\bar{x} = \frac{10 + 10 + 10 + 10 + 60}{5} = \frac{100}{5} = 20$$

Step 2 · Calculate the median. The values are already sorted. With five values, the median is the third value: $\text{median}(x) = 10$.

Your numerical answer is correct. One small refinement: rather than saying the distribution is "not normal," say it is **right-skewed** in this example. The unusually large value, 60, pulls the mean upward, while the median remains 10.



Why the mean shifts: one outlier pulls it to 20.

Four values are 10, but the value 60 raises the average. This is why the median is often a **more robust** measure of the center when outliers are present.

```
np.mean(x)    # 20.0
np.median(x)  # 10.0
```

Exercise 4 · Correlation CORRECT

Your answer: 1, −1.

- `a` and `b` have a **perfect positive** linear relationship. Every value of `b` is twice the corresponding value of `a`, so $r = 1$.
- `a` and `c` have a **perfect negative** linear relationship. As `a` increases by 1, `c` decreases by 2, so $r = -1$.

You can verify this using NumPy:

```
np.corrcoef(a, b)[0, 1]  # 1.0
np.corrcoef(a, c)[0, 1]  # -1.0
```

> Remember that the correlation coefficient measures the direction and strength of a **linear** relationship. A correlation of zero does not necessarily mean that two variables have no relationship at all; they could have a nonlinear relationship.

Exercise 5 · NumPy and pandas ONE FIX NEEDED

Your answer is conceptually correct, with one Python function to fix. You wrote:

```
np.mean(x)
np.var(x)
np.std(x)
np.corr(x, y)
```

The first three functions are correct. However, `np.corr()` is not the standard NumPy function for calculating Pearson correlation. Use:

```
np.corrcoef(x, y)[0, 1]
```

Here's a complete example:

```
import numpy as np
import pandas as pd

x = np.array([1, 2, 3, 4])
y = np.array([2, 4, 6, 8])

mean_x = np.mean(x)
variance_x = np.var(x)
std_x = np.std(x)
correlation = np.corrcoef(x, y)[0, 1]

print(mean_x)       # 2.5
print(variance_x)   # 1.25
print(std_x)        # 1.118...
print(correlation)  # 1.0
```

Your explanation about sample versus population statistics is also correct. Let's make the distinction precise.

Operation	NumPy default	pandas Series default
Variance	`np.var(x)` → population, divides by N	`pd.Series(x).var()` → sample, divides by $N-1$
Standard deviation	`np.std(x)` → population	`pd.Series(x).std()` → sample

For example:

```
np.var(x)                 # 1.25
pd.Series(x).var()        # 1.6667...
np.var(x, ddof=1)         # 1.6667...
```

`ddof` means **delta degrees of freedom**. The denominator is $N - \text{ddof}$:

- `ddof=0`: divide by N.
- `ddof=1`: divide by $N - 1$.

The important point is that pandas' defaults differ from NumPy's for variance and standard deviation, but you can make them consistent by setting `ddof` explicitly.

Final assessment

- **Exercise 1:** Correct.
- **Exercise 2:** Correct final answer; reverse the signs in your deviation list.
- **Exercise 3:** Correct calculations; describe the data as right-skewed rather than concluding that it is not normal.
- **Exercise 4:** Both correlations correct.
- **Exercise 5:** Correct statistical understanding; replace `np.corr()` with `np.corrcoef(x, y)[0, 1]`.

You understand the key concepts well. Your main next step is translating that understanding into exact Python APIs and precise statistical language.

Module 5 · Derivatives, Gradients & Partial Derivatives

Topic: Derivatives, gradients, partial derivatives · **Output:** gradient exercises

Before solving these, remember the three ideas:

- **Derivative:** how fast a function changes with respect to one variable.
- **Partial derivative:** how a function of multiple variables changes when you vary one variable and hold the others constant.
- **Gradient:** the vector of all partial derivatives. It points in the direction of steepest increase for a differentiable function.

For example, if $f(x, y) = x^2 + 3y$, then:

$$\nabla f(x, y) = \begin{bmatrix} 2x \\ 3 \end{bmatrix}$$

Now let's practice, starting with basic derivatives and moving toward gradients used in machine learning.

Exercises

Exercise 1 · Basic derivative

Given $f(x) = 3x^2 + 2x - 5$, find $f'(x)$. Then calculate $f'(2)$.

Exercise 2 · Derivative and slope

Given $f(x) = x^2$, find the derivative and the slope of the curve at $x = 3$. Is the function increasing or decreasing at that point?

Exercise 3 · Partial derivatives

Given $f(x, y) = 3x^2 + 2xy + y^2$, find $\partial f/\partial x$ and $\partial f/\partial y$.

Exercise 4 · Evaluate the gradient

For $f(x, y) = x^2 + 4y^2$, calculate $\nabla f(2, -1)$.

Exercise 5 · Gradient in three dimensions

Given $f(x, y, z) = xy + yz + x^2$, find $\nabla f(x, y, z)$.

Exercise 6 · Gradient descent

Let $f(x) = x^2$ and start at $x = 4$. Using gradient descent with learning rate $\alpha = 0.1$, calculate the gradient at $x = 4$ and the next value: $x_{\text{new}} = x - \alpha f'(x)$.

Exercise 7 · ML loss function

A model has prediction $\hat{y} = wx$ and squared-error loss $L(w) = (wx - y)^2$. Treat x and y as constants. Find $\partial L/\partial w$.

Suggested approach: show your working rather than writing only the final answer. In particular, pay attention to which variables are held constant in partial derivatives and how the gradient is used in Exercise 6. Those are the foundations for understanding backpropagation later.

YOUR SUBMITTED ANSWERS

1. $6x + 2 \Rightarrow 14$
2. $2x \Rightarrow 6$, increasing
3. $x \Rightarrow 6x + 2y$; $y \Rightarrow 2x + 2y$
4. $x \Rightarrow 2x \Rightarrow 4$; $y \Rightarrow 8y \Rightarrow -8$
5. $x \Rightarrow y + 2x$; $y \Rightarrow x + z$; $z \Rightarrow y$
6. $2x \Rightarrow 8 \Rightarrow 4 - 0.1 \times 8 = 3.2$
7. $L(w) = w^2x^2 - 2wxy + y^2 \Rightarrow \partial L/\partial w = 2wx^2 - 2xy$

Review and corrections

Excellent work: all seven answers are mathematically correct. Exercise 7 is especially good because you expanded the loss function and differentiated it correctly. Let's check each answer step by step and connect the calculations to machine learning.

Score: 7/7. All answers correct.

Exercise 1 · Basic derivative CORRECT

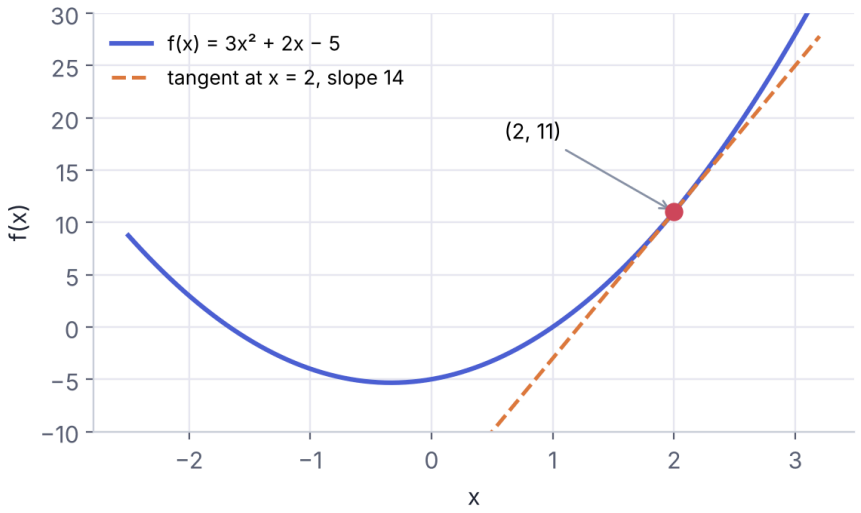
Given $f(x) = 3x^2 + 2x - 5$. Your answer: $f'(x) = 6x + 2$, so $f'(2) = 14$.

Step by step:

1. The **power rule** says $\frac{d}{dx}x^n = nx^{n-1}$.
2. Therefore, $\frac{d}{dx}(3x^2) = 6x$.
3. $\frac{d}{dx}(2x) = 2$.
4. The derivative of a constant is zero: $\frac{d}{dx}(-5) = 0$.

So $f'(x) = 6x + 2$. Substitute $x = 2$:

$$f'(2) = 6(2) + 2 = \boxed{14}$$



The tangent line at x = 2 has slope 14.

Concept to remember: a derivative tells you the **instantaneous rate of change** of a function. Here, at $x = 2$, the function's slope is 14.

Exercise 2 · Derivative and slope CORRECT

Given $f(x) = x^2$. Your answer: $f'(x) = 2x$, slope $= 6$, increasing.

$$f'(x) = 2x \quad \Rightarrow \quad f'(3) = 2(3) = \boxed{6}$$

Because the derivative is positive at $x = 3$, the function is increasing at that point.

- Positive derivative → increasing locally.
- Negative derivative → decreasing locally.
- Zero derivative → horizontal tangent; it may be a maximum, minimum, or another stationary point.

Notice that the **derivative is a function** ($2x$), while the **slope at a particular point is a number** (6).

Exercise 3 · Partial derivatives CORRECT

Given $f(x, y) = 3x^2 + 2xy + y^2$. Your answers: with respect to x: $6x + 2y$; with respect to y: $2x + 2y$.

The key idea is that a partial derivative measures how a function changes with respect to **one** variable while treating the other variables as **constants**.

1. Differentiate with respect to x (treat y as a constant):

- $3x^2 \to 6x$
- $2xy \to 2y$, because y is constant.
- $y^2 \to 0$, because it contains no x.

$$\boxed{\frac{\partial f}{\partial x} = 6x + 2y}$$

2. Differentiate with respect to y (treat x as a constant):

- $3x^2 \to 0$
- $2xy \to 2x$
- $y^2 \to 2y$

$$\boxed{\frac{\partial f}{\partial y} = 2x + 2y}$$

AI CONNECTION

Why this matters in ML: a model can have thousands of parameters. Partial derivatives tell us how changing one parameter affects the loss while the other parameters are held fixed.

Exercise 4 · Evaluate the gradient CORRECT

Given $f(x, y) = x^2 + 4y^2$. Your answer: the x-component is 4, and the y-component is -8.

First, find both partial derivatives:

$$\frac{\partial f}{\partial x} = 2x \qquad \frac{\partial f}{\partial y} = 8y$$

The gradient is the vector containing all the partial derivatives:

$$\nabla f(x, y) = \begin{bmatrix} 2x \\ 8y \end{bmatrix}$$

Evaluate it at $(x, y) = (2, -1)$:

$$\nabla f(2, -1) = \begin{bmatrix} 2(2) \\ 8(-1) \end{bmatrix} = \boxed{\begin{bmatrix} 4 \\ -8 \end{bmatrix}}$$

The negative sign matters: because $y = -1$, the second component is -8, not $+8$.

Intuition: the gradient points in the direction of the steepest local increase of the function. Its components tell you how sensitive the function is to changes in each variable.

Exercise 5 · Gradient in three dimensions CORRECT

Given $f(x, y, z) = xy + yz + x^2$. Your answers: $\frac{\partial f}{\partial x} = y + 2x$, $\frac{\partial f}{\partial y} = x + z$, $\frac{\partial f}{\partial z} = y$.

Check each variable independently:

Variable	Terms that change	Partial derivative
x	$xy,\ x^2$	$y + 2x$
y	$xy,\ yz$	$x + z$
z	yz	y

So the full gradient is:

$$\boxed{\nabla f(x, y, z) = \begin{bmatrix} y + 2x \\ x + z \\ y \end{bmatrix}}$$

This is exactly the same process as Exercise 4, except now there are three variables and three partial derivatives. In a real ML model, the gradient might have **millions of components**, one for each trainable parameter.

Exercise 6 · Gradient descent CORRECT

Given $f(x) = x^2$, $x = 4$, $\alpha = 0.1$. Your answer: gradient $= 8$, next value $= 3.2$.

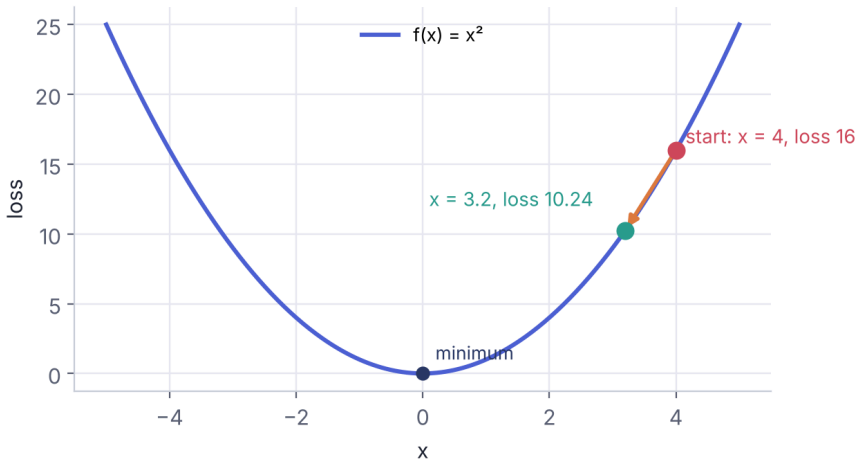
Step 1 · Calculate the derivative: $f'(x) = 2x$

Step 2 · Calculate the gradient at the current point: $f'(4) = 2(4) = \boxed{8}$

Step 3 · Apply the gradient descent formula:

$$x_{\text{new}} = x - \alpha f'(x) = 4 - 0.1(8) = 4 - 0.8 = \boxed{3.2}$$

What just happened? The starting point $x = 4$ has loss 16; the new point $x = 3.2$ has loss 10.24.



One gradient descent step moves x from 4 to 3.2.

The update moves x toward zero, where x^2 reaches its minimum. The loss decreases from 16 to 10.24.

Understand the formula:

$$\underbrace{x_{\text{new}}}_{\text{updated parameter}} = \underbrace{x}_{\text{current parameter}} - \underbrace{\alpha}_{\text{learning rate}} \underbrace{f'(x)}_{\text{gradient}}$$

- **Parameter x:** what we're trying to optimize.
- **Gradient $f'(x)$:** the direction and rate of increase.
- **Learning rate α:** how large a step we take.
- **Minus sign:** moves against the gradient to reduce the function locally.

If the learning rate is too large, updates can overshoot the minimum or even diverge. If it is too small, learning can be slow.

Exercise 7 · ML loss function and chain rule CORRECT

Given $L(w) = (wx - y)^2$. You expanded it as $L(w) = w^2x^2 - 2wxy + y^2$, then calculated $\frac{\partial L}{\partial w} = 2wx^2 - 2xy$.

Your expansion and differentiation are both right. Let's also solve it using the **chain rule**, because that is the more reusable approach in machine learning.

Method 1 · Expand first. Since x and y are constants, differentiate each term with respect to w:

$$L(w) = w^2x^2 - 2wxy + y^2 \quad \Rightarrow \quad \frac{\partial L}{\partial w} = 2wx^2 - 2xy + 0$$

$$\boxed{\frac{\partial L}{\partial w} = 2wx^2 - 2xy}$$

Factor out $2x$:

$$\boxed{\frac{\partial L}{\partial w} = 2x(wx - y)}$$

Both forms are equivalent.

Method 2 · Use the chain rule. The chain rule is useful when one function is nested inside another. Define the inner expression $u = wx - y$; then the loss is $L = u^2$.

- **Step 1 · Differentiate the outer function:** $\frac{dL}{du} = 2u$
- **Step 2 · Differentiate the inner expression with respect to w:** $\frac{\partial u}{\partial w} = x$ (remember that x is constant).
- **Step 3 · Multiply the derivatives:**

$$\frac{\partial L}{\partial w} = \frac{dL}{du}\frac{\partial u}{\partial w} = 2u \cdot x = 2(wx - y)x$$

$$\boxed{\frac{\partial L}{\partial w} = 2x(wx - y)}$$

Derivative as a limit: the slope at a point is the limit of the slope of secant lines, $f'(a) = \lim_{h \to 0} \frac{f(a+h)-f(a)}{h}$. For example, for $f(x) = x^2$ at $a = 1$ with $h = 2$, the secant slope is $\frac{f(3)-f(1)}{2} = 4$, while the true slope is 2 (error $= 2$). As h shrinks, the error shrinks too. The chain rule lets us calculate that slope with respect to the parameter w, even though the loss is built from nested operations.

Why this matters in ML. In a simple linear model, $\hat{y} = wx$, we want to adjust w so the prediction approaches the true value y.

- If $wx - y$ is positive, the prediction is too high.
- If $wx - y$ is negative, the prediction is too low.
- The gradient combines this error with the input x, telling us how the loss changes when we adjust w.

Gradient descent then updates the parameter:

$$\boxed{w_{\text{new}} = w - \alpha \frac{\partial L}{\partial w}} \quad \Rightarrow \quad w_{\text{new}} = w - \alpha\, 2x(wx - y)$$

This is the core idea behind training many ML models: **calculate the loss, differentiate it with respect to the parameters, and update those parameters to reduce the loss.**

Final review

Exercise	Result	Key skill
1	Correct	Power rule
2	Correct	Derivative as slope
3	Correct	Partial derivatives
4	Correct	Gradient at a point
5	Correct	Multivariable gradient
6	Correct	Gradient descent update
7	Correct	Chain rule and loss gradients

Score: 7/7. You have the foundational calculations down. The next important step is to understand how these ideas combine in **backpropagation**: using the chain rule repeatedly to calculate gradients throughout a neural network.

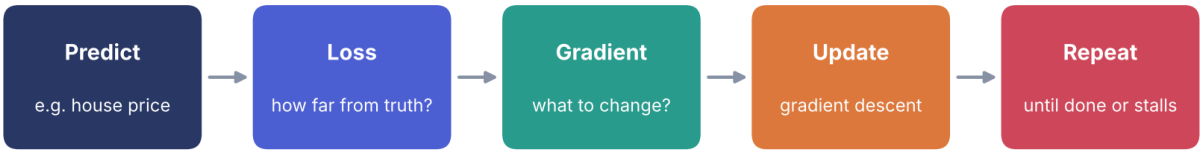
Module 6 · Optimization Notebook: Loss Functions, Learning Rate & Gradient Descent

Topic: Loss functions, learning rate, gradient descent · **Output:** optimization notebook

This lesson builds directly on derivatives and gradients. We'll use Python and NumPy to understand how a model learns by reducing its loss.

By the end, you should be able to calculate a loss, implement gradient descent, understand the learning rate, and recognize when optimization is going wrong.

1. The big picture



The training loop.

1. **Model makes a prediction**: for example, predict a house price.
2. **Calculate the loss**: how far is the prediction from the true answer?
3. **Calculate the gradient**: how should we change the model's parameters?
4. **Update the parameters**: use gradient descent to reduce the loss.
5. **Repeat** until training is complete or progress stalls.

> **The important distinction:** the **loss** measures how bad the prediction is, while the **gradient** tells us how to change the parameters to improve it.

2. Loss functions

A **loss function** converts a prediction error into a numerical score. For regression, two common choices are:

Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

- $\hat{y}_i$: predicted value.
- y_i: actual value.
- n: number of examples.

Squaring makes errors positive and penalizes large errors more heavily.

Example: actual values are $[3, 5]$, predictions are $[2, 7]$.

$$\text{MSE} = \frac{(2-3)^2 + (7-5)^2}{2} = \frac{1+4}{2} = \boxed{2.5}$$

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|$$

For the same predictions:

$$\text{MAE} = \frac{|2-3| + |7-5|}{2} = \boxed{1.5}$$

MSE is more sensitive to large errors; MAE is less affected by outliers. The appropriate loss depends on the task and what kinds of errors matter.

3. Gradient descent

Suppose we want to minimize a simple function:

$$f(w) = w^2$$

Its minimum occurs at $w = 0$, where the loss is zero. Its derivative is $f'(w) = 2w$. Gradient descent updates the parameter by moving in the **opposite** direction of the gradient:

$$\boxed{w_{\text{new}} = w - \alpha f'(w)}$$

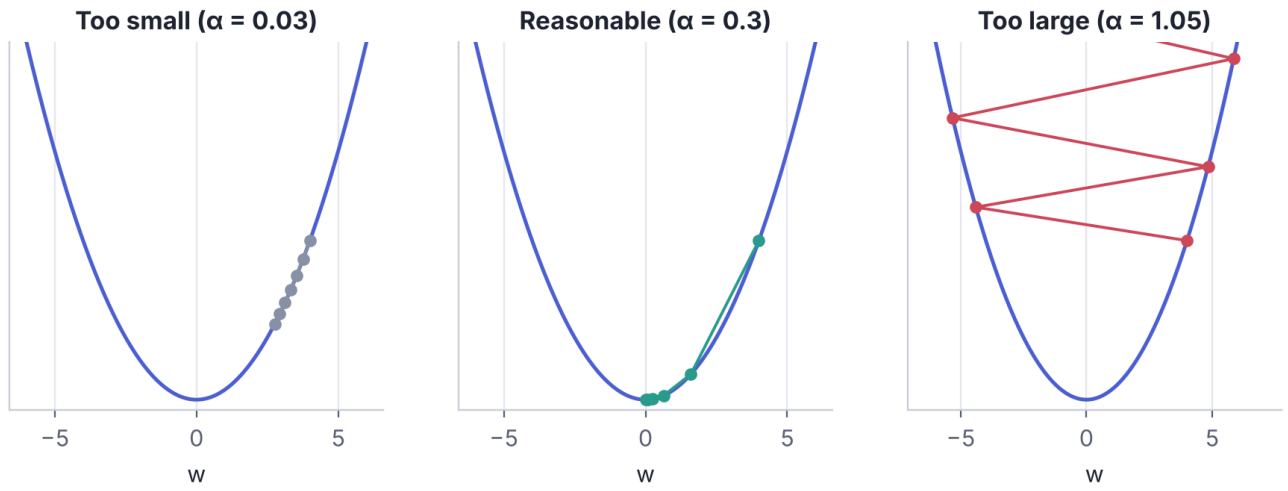
Here, α is the **learning rate**. Let's start at $w = 4$ with $\alpha = 0.1$.

- Current loss: $4^2 = 16$.
- Gradient: $2(4) = 8$.
- Update: $4 - 0.1(8) = 3.2$.
- New loss: $3.2^2 = 10.24$.

The loss decreased from 16 to 10.24.

4. Learning rate: why it matters

The learning rate determines the size of each optimization step.



Small, reasonable, and too-large learning rates on the same loss curve.

Learning rate	What happens
Too small	Updates make progress, but training can take a long time.
Reasonable	Updates can move toward the minimum efficiently.
Too large	Updates may overshoot the minimum, oscillate, or diverge.

For $f(w) = w^2$, the update becomes:

$$w_{\text{new}} = w - 2\alpha w = (1 - 2\alpha)w$$

This tells us exactly how the learning rate affects this example:

- $0 < \alpha < 0.5$: the parameter converges smoothly toward zero.
- $\alpha = 0.5$: it reaches zero in one update.
- $0.5 < \alpha < 1$: the parameter overshoots and alternates sides of zero, but its size still shrinks, so it converges.
- $\alpha = 1$: the parameter bounces between $+w$ and $-w$ forever.
- $\alpha > 1$: the parameter overshoots and grows; the updates diverge.

> **Correction:** the original lesson said every $\alpha > 0.5$ diverges for this function. That was corrected later in the session (see the answer review below): the updates only diverge when $\alpha > 1$. The list above uses the correct ranges. These thresholds are specific to this function, not universal rules for every ML model.

5. Your Optimization Notebook

Run these cells in a Jupyter notebook. Try to understand each operation rather than just copying the output.

Cell 1 · Calculate loss functions

```python
import numpy as np

y_true = np.array([3, 5])
y_pred = np.array([2, 7])

mse = np.mean((y_true - y_pred) ** 2)
mae = np.mean(np.abs(y_true - y_pred))

print("MSE:", mse)
print("MAE:", mae)
```

Expected output:

```
MSE: 2.5
MAE: 1.5
```

Cell 2 · Implement gradient descent from scratch

```python
def loss(w):
    return w ** 2

def gradient(w):
    return 2 * w

w = 4.0
learning_rate = 0.1

for step in range(10):
    current_loss = loss(w)
    grad = gradient(w)

    w = w - learning_rate * grad

    print(
        f"Step {step + 1}: "
        f"w={w:.4f}, loss={loss(w):.4f}"
    )
```

Notice the order:

1. Calculate the gradient at the current w.
2. Update w.
3. Calculate the loss at the updated w.

The parameter and loss should approach zero:

```
Step 1: w=3.2000, loss=10.2400
Step 2: w=2.5600, loss=6.5536
Step 3: w=2.0480, loss=4.1943
Step 4: w=1.6384, loss=2.6844
Step 5: w=1.3107, loss=1.7180
Step 6: w=1.0486, loss=1.0995
Step 7: w=0.8389, loss=0.7037
Step 8: w=0.6711, loss=0.4504
Step 9: w=0.5369, loss=0.2882
Step 10: w=0.4295, loss=0.1845
```

Cell 3 · Compare learning rates

```python
learning_rates = [0.01, 0.1, 0.5, 0.9]

for lr in learning_rates:
    w = 4.0

    for _ in range(10):
        w = w - lr * 2 * w

    print(f"lr={lr}: w={w:.6f}, loss={w**2:.6f}")
```

Output:

```
lr=0.01: w=3.268291, loss=10.681728
lr=0.1: w=0.429497, loss=0.184467
lr=0.5: w=0.000000, loss=0.000000
lr=0.9: w=0.429497, loss=0.184467
```

Think about why `0.01` makes slower progress than `0.1`, and why `0.9` behaves differently from both. (Hint: with `0.9` the multiplier is $1 - 2(0.9) = -0.8$, so after an even number of steps it lands on exactly the same value as `0.1`, whose multiplier is $+0.8$; it just zig-zags across zero to get there.)

6. Exercises

Answer in your own words or show your calculations. Partial answers are fine.

Exercise 1 · MSE

Actual values are `[2, 4, 6]` and predictions are `[3, 2, 8]`. Calculate MSE by hand.

Exercise 2 · MAE vs MSE

Errors have magnitudes 1 and 5. Calculate MAE and MSE. Which one penalizes the larger error more strongly?

Exercise 3 · One update

For $f(w) = w^2$, start at $w = 3$ with learning rate 0.1. What are the gradient, new w, and new loss?

Exercise 4 · Learning rate

For $f(w) = w^2$, start at $w = 4$ and use learning rate 0.6. Calculate the first updated w and loss. Is the loss lower or higher than before?

Exercise 5 · Spot the bug

A developer writes `w = w + learning_rate * gradient` for minimizing a loss. What is wrong with this update?

Exercise 6 · Code interpretation

In Cell 2, why must we calculate `gradient(w)` repeatedly instead of calculating it only once before the loop?

YOUR SUBMITTED ANSWERS

1. `3`
2. MSE: $(1 + 25)/2 = 13$; MAE: $(1 + 5)/2 = 3$. MSE shows it more strongly.
3. New $w = w - 0.1 \times 2w$. Gradient: $2w$. New w: 2.4. New loss: $2.4 \times 2 = 4.8$
4. Loss: 8. New w: $4 - 0.6 \times 8 = -0.8$. New loss $= -1.6$. The loss is lower than before (only the amount), but because the learning rate was too high we passed the ideal value and need to come back again.
5. The issue is with minimizing the loss. We are not doing gradient for that; we just do the gradient.
6. Because it repeatedly gets changed: both the x/y values and the w/b values.

7. Answer review

You're doing well with the concepts, especially in Exercise 2 and your intuition about overshooting in Exercise 4. The main things to tighten up are **calculating loss from the updated parameter** and **distinguishing the gradient from the loss itself**.

Exercise 1 · MSE CORRECT

Actual values $[2, 4, 6]$, predictions $[3, 2, 8]$. Your answer: 3.

MSE is the average of the squared differences between each prediction and its actual value.

Step 1 · Calculate the errors:

$$\hat{y} - y = [3-2,\ 2-4,\ 8-6] = [1, -2, 2]$$

Step 2 · Square each error:

$$[1^2, (-2)^2, 2^2] = [1, 4, 4]$$

Step 3 · Calculate the average:

$$\text{MSE} = \frac{1+4+4}{3} = \boxed{3}$$

Your answer is correct.

> **Editor's note:** the original review first labeled this answer "Incorrect" and then confirmed it was correct. That badge was a mistake; MSE is 3.

Remember: negative errors become positive after squaring, so overpredictions and underpredictions both contribute to the loss.

Exercise 2 · MAE vs. MSE CORRECT

Errors have magnitudes 1 and 5. Your answers: MSE $= 13$, MAE $= 3$, MSE penalizes the larger error more strongly. You got all three right.

$$\text{MSE} = \frac{1^2 + 5^2}{2} = \frac{1+25}{2} = \boxed{13} \qquad \text{MAE} = \frac{|1| + |5|}{2} = \frac{1+5}{2} = \boxed{3}$$

Why does MSE penalize the larger error more strongly? Because squaring makes large errors grow faster than small ones. Here, an error of 5 contributes 25 to the squared-error sum, while an error of 1 contributes only 1.

A useful distinction: MSE and MAE have different numerical scales, so comparing their raw values alone is not always enough. The important point is how each function responds to errors.

Exercise 3 · One gradient descent update ALMOST CORRECT

Given $f(w) = w^2$, $w = 3$, $\alpha = 0.1$. Your answer: gradient $2w$, new $w = 2.4$, new loss $= 2.4 \times 2 = 4.8$.

Your gradient formula and updated parameter are correct. The mistake is in the final loss calculation.

Step 1 · Find the gradient: $f'(w) = 2w$, so at $w = 3$: $f'(3) = 2(3) = 6$.

Step 2 · Update the parameter:

$$w_{\text{new}} = w - \alpha f'(w) = 3 - 0.1(6) = \boxed{2.4}$$

Step 3 · Calculate the new loss. The loss function is $f(w) = w^2$, so substitute $w = 2.4$:

$$f(2.4) = (2.4)^2 = \boxed{5.76}$$

You multiplied 2.4 by 2, which calculates the **gradient** at 2.4, not the **loss**.

> **Keep these two calculations separate:**
>
> - **Loss:** $f(w) = w^2$
> - **Gradient:** $f'(w) = 2w$

The original loss was $3^2 = 9$, and the new loss is 5.76. Therefore, this update successfully reduced the loss.

Exercise 4 · Learning rate and overshooting PARTIALLY CORRECT

Given $f(w) = w^2$, $w = 4$, $\alpha = 0.6$. Your answer: gradient $= 8$, new $w = -0.8$, new loss $= -1.6$; you recognized that the update passed the minimum.

Your parameter update and intuition about overshooting are correct. Let's fix the loss calculation.

Step 1 · Calculate the gradient: $f'(4) = 2(4) = 8$

Step 2 · Update the parameter:

$$w_{\text{new}} = 4 - 0.6(8) = \boxed{-0.8}$$

The negative result means the update crossed the minimum at $w = 0$.

Step 3 · Calculate the new loss:

$$f(-0.8) = (-0.8)^2 = \boxed{0.64}$$

The loss cannot be -1.6, because $f(w) = w^2$ is never negative. The original loss was 16, so the loss is still **lower** after this update. (Also note that the starting loss is $4^2 = 16$; the 8 you wrote is the gradient.)



Visualize the overshoot: from w = 4 to w = −0.8.

The minimum is at $w = 0$. Your update moves from 4 to -0.8, crossing the minimum but landing at a point with much lower loss.

> **An important correction to the earlier lesson:** for this specific function, a learning rate of 0.6 does **not** cause divergence. It causes the parameter to alternate sides of zero while converging. In fact, $w_{\text{new}} = (1 - 2\alpha)w$, and for this quadratic, convergence occurs when $0 < \alpha < 1$. At $\alpha = 0.6$, the multiplier is -0.2, so the parameter flips sign and becomes smaller in magnitude each step.

Your intuition about overshooting was right; the key is that overshooting does not automatically mean the loss increases or that training diverges.

Exercise 5 · Spot the bug NEEDS CLARIFICATION

The developer writes `w = w + learning_rate * gradient`. Your answer: the issue is with minimizing the loss; we should use the gradient.

You're right that the problem concerns minimizing the loss, but the gradient is exactly what we need. **The bug is the plus sign instead of the minus sign.** The correct gradient descent update is:

```
w = w - learning_rate * gradient
```

Why? Imagine you're standing on a hill:

- The gradient points **uphill**, toward increasing loss.
- To minimize the loss, we generally move in the opposite direction.
- The minus sign tells us to move **against** the gradient.

For example, if $w = 3$, then $f'(w) = 6$. With learning rate 0.1:

	Update	New w	New loss
Correct update	$3 - 0.1(6)$	2.4	$f(2.4) = 5.76$
Buggy update	$3 + 0.1(6)$	3.6	$f(3.6) = 12.96$

The correct update reduces the loss from 9 to 5.76. The buggy update increases it from 9 to 12.96. The plus-sign version is called **gradient ascent** when used to maximize an objective.

Exercise 6 · Why recalculate the gradient? CORRECT IDEA

Your answer: because it repeatedly changes, including x, y, and w.

You've identified the key idea: **the gradient changes when the parameter changes.** In this particular example, only w is being optimized; there are no x and y variables in the function. The loss and gradient are:

$$f(w) = w^2, \qquad f'(w) = 2w$$

Suppose we start at $w = 4$ and use learning rate 0.1.

Step	Current w	Gradient $2w$	Updated w
1	4.00	8.00	3.20
2	3.20	6.40	2.56
3	2.56	5.12	2.048

Each update changes w. Since the gradient is $2w$, its value changes too. If you calculated the gradient only once at the beginning and reused 8, the algorithm would keep taking the same-sized step, even after the parameter moved. That would no longer be the correct gradient descent update for the current position.

In a neural network, the same principle applies: **after parameters are updated, gradients must be recalculated for the next step.**

Your optimization progress

Skill	Assessment
MSE and MAE	Strong
Gradient descent update	Correct formula; practice evaluating the new loss
Learning-rate intuition	Good, especially recognizing overshooting
Loss vs. gradient	Review this distinction
Why gradients are recalculated	Good understanding
Correct update direction	Now clarified

The main pattern to practice:

1. Calculate the gradient at the current parameter.
2. Update the parameter using the learning rate.
3. Evaluate the **original loss function** at the new parameter.
4. Compare the old and new losses.

That distinction between the gradient and the loss is essential before moving on to optimizing models with multiple parameters.

8. Optimization retest: aiming for 100/100

You asked for another test so you could score 100/100. This retest used fresh questions focused on the mistakes from the last round: calculating loss correctly, applying the learning rate, handling overshooting, and choosing the right gradient update.

Result: perfect score. You got all 6 questions right.

Question 1. Actual values are `[1, 4, 6]` and predictions are `[3, 2, 7]`. What is the mean squared error (MSE)?

Answer: 3. The errors are 2, -2, and 1. Squaring gives 4, 4, and 1; their mean is $9/3 = 3$.

Question 2. For $f(w) = w^2$, the current parameter is $w = 3$ and the learning rate is 0.1. After one gradient descent update, what is the new loss?

Answer: 5.76. The gradient at 3 is 6, so the updated parameter is $3 - 0.1 \times 6 = 2.4$. The loss there is $(2.4)^2 = 5.76$.

Question 3. For $f(w) = w^2$, start at $w = 4$ with learning rate 0.6. What are the parameter after one update and its loss?

Answer: $w = -0.8$, loss $= 0.64$. The gradient is 8, so the update gives $4 - 0.6 \times 8 = -0.8$. Squaring it gives a loss of 0.64.

Question 4. For $f(w) = w^2$, what happens after one gradient descent update from $w = 4$ with learning rate 1.2?

Answer: the loss increases. The update is $4 - 1.2 \times 8 = -5.6$, and the squared loss is 31.36, which is greater than the original loss 16.

Question 5. For a gradient descent update $w_{\text{new}} = w - \alpha g$, where $\alpha > 0$ and g is the current gradient, which statements are correct? Select every correct answer.

Answer: If g is positive, the update moves w downward. If g is negative, the update moves w upward.

Question 6. For $f(w) = w^2$, after updating w, how are the loss and the next gradient calculated?

Answer: The loss is calculated by evaluating f at the new w, i.e. $f(w_{\text{new}})$, while the gradient for the next step is calculated by evaluating f' at that new w, i.e. $f'(w_{\text{new}})$. Both are evaluated using the updated parameter.

Module 7 · Linear Regression from Scratch

Topic: Linear regression from scratch · **Output:** NumPy model

Let's move to the next topic in your AI Engineering roadmap: **Linear Regression from Scratch using NumPy**. Our goal is to understand how a machine learning model learns from data, then implement it ourselves without using scikit-learn.

1. What is linear regression?

Imagine you have data about house sizes and their prices. You want a model that predicts a house's price from its size. The model tries to find a line that fits the observed data as well as possible. Statisticians often write this line as $\hat{y} = b_0 + b_1 x$; in machine learning we usually write it with a weight and a bias.

For one input feature, our model is:

$$\hat{y} = wx + b$$

- x: input feature, such as house size.
- $\hat{y}$: predicted output, such as house price.
- w: **weight**, or slope of the line.
- b: **bias**, or intercept.
- y: actual output from the dataset.

The model starts with values for w and b, then learns better values by minimizing its prediction errors.

2. How does the model learn?

We use the same optimization concepts you just studied: loss, gradients, and gradient descent.



The four-step learning loop for linear regression.

1. **Make predictions:** use the current w and b to calculate predicted values.
2. **Calculate the loss:** use Mean Squared Error (MSE) to measure how far predictions are from actual values.
3. **Calculate gradients:** determine how changing the weight and bias affects the loss.
4. **Update parameters:** apply gradient descent and repeat until the model learns a good fit.

The loss function

For n training examples:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

The model wants to find w and b that minimize this value.

The gradients

For our model $\hat{y} = wx + b$, the gradients are:

$$\frac{\partial MSE}{\partial w} = \frac{2}{n}\sum_{i=1}^{n} x_i(\hat{y}_i - y_i)$$

$$\frac{\partial MSE}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)$$

Notice the connection to your previous lesson: the gradients tell us how the loss changes with respect to each parameter. (Compare with Module 5, Exercise 7, where $\partial L/\partial w = 2x(wx - y)$ for a single example.)

3. Implement linear regression with NumPy

We'll use a tiny dataset where the relationship is $y = 2x + 1$. That means the ideal weight is 2 and the ideal bias is 1.

```python
import numpy as np

# 1. Training data
X = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([3, 5, 7, 9, 11], dtype=float)

# 2. Initialize parameters
w = 0.0
b = 0.0
learning_rate = 0.01
epochs = 2000
n = len(X)

# 3. Train the model
for epoch in range(epochs):
    # Predictions
    y_pred = w * X + b

    # Calculate loss
    loss = np.mean((y_pred - y) ** 2)

    # Calculate gradients
    dw = (2 / n) * np.sum(X * (y_pred - y))
    db = (2 / n) * np.sum(y_pred - y)

    # Update parameters
    w = w - learning_rate * dw
    b = b - learning_rate * db

# 4. Show the trained model
print("Weight:", w)
print("Bias:", b)
print("Final MSE:", np.mean((w * X + b - y) ** 2))

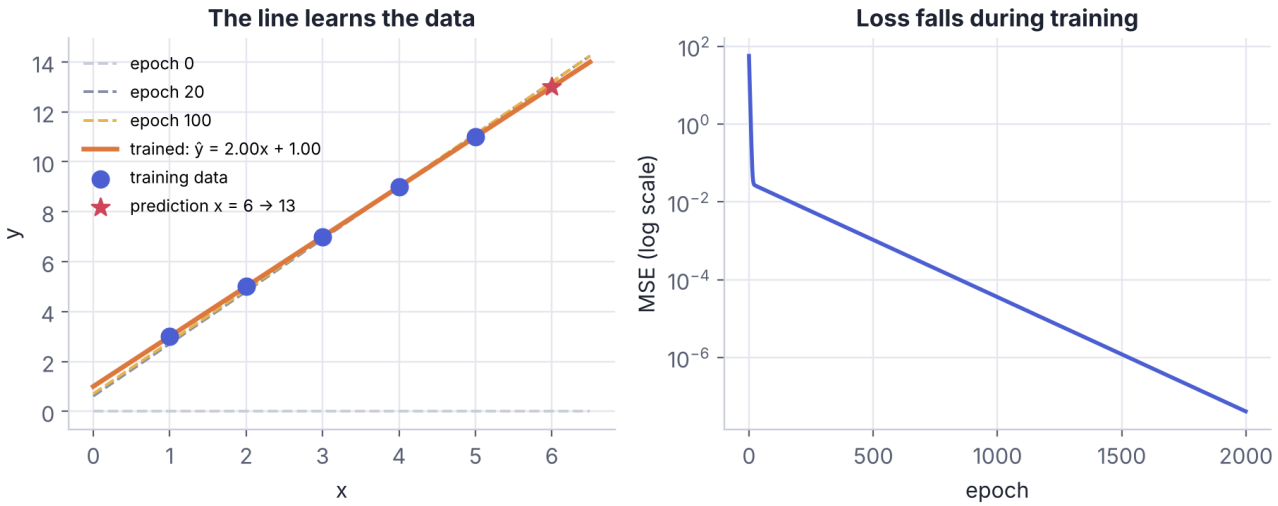
# 5. Predict for a new input
new_x = 6
prediction = w * new_x + b
print("Prediction for x=6:", prediction)
```

4. What output should you expect?

After training, the parameters should be close to:

```
Weight: 2.0
Bias: 1.0
Final MSE: approximately 0.0
Prediction for x=6: approximately 13.0
```

The exact printed values may have tiny floating-point differences (an actual run gives `Weight: 2.00013`, `Bias: 0.99953`, `Final MSE: 4.1e-08`, `Prediction: 13.0003`).



Left: the line converging to the data. Right: the MSE dropping over 2,000 epochs.

The learned model is approximately:

$$\hat{y} = 2x + 1$$

So for $x = 6$, the prediction is $2(6) + 1 = 13$.

5. Check your understanding

Question 1. What does `dw` represent?

a. The current prediction

b. The gradient of the loss with respect to the weight

c. The number of training examples

Question 2. Why do we update `w` and `b` after calculating the gradients?

a. To move the parameters toward values that reduce the loss

b. To increase the number of data points

c. To make the predictions equal to the inputs

Question 3. If `w = 2`, `b = 1`, and `x = 4`, what is the prediction?

a. 7

b. 8

c. 9

YOUR SUBMITTED ANSWERS: Q1 = b, Q2 = a, Q3 = c

6. Review of your answers

Excellent: **3/3 correct! Score: 100/100.**

Score: 3/3 (100/100).

Q1: B CORRECT. `dw` is the gradient of the loss with respect to the weight w. It tells us how the loss changes when the weight changes.

Q2: A CORRECT. We update the weight and bias using gradient descent to move toward parameters that reduce the loss. The learning rate controls the size of each update.

Q3: C CORRECT. Substitute $w = 2$, $b = 1$, and $x = 4$:

$$\hat{y} = wx + b = 2(4) + 1 = 9$$

You understand the core concepts. The next step is to make your implementation reusable, just like a real machine-learning library: build a `LinearRegression` class in NumPy with `fit(X, y)` and `predict(X)` methods, then test it on new data.

Module 8 · Gradient Descent from Scratch: Comparing Learning Rates

Topic: Gradient descent from scratch · **Output:** comparing learning rates

Let's build on what you've learned about gradients, loss functions, and linear regression. This time, we'll implement gradient descent from scratch with NumPy and compare different learning rates.

By the end, you'll understand how the learning rate affects convergence, how to detect divergence, and how to visualize the results.

1. The idea

Gradient descent repeatedly adjusts a parameter to reduce a loss function.



The optimization loop.

1. Start with a parameter w.
2. Calculate the gradient $f'(w)$.
3. Update the parameter: $w \leftarrow w - \alpha f'(w)$.
4. Repeat and track the loss.

Here, α (alpha) is the **learning rate**: it controls how large each update is.

We'll use the simple loss function $f(w) = w^2$. Its minimum is at $w = 0$, where the loss is zero, and its gradient is $2w$.

2. Compare three learning rates

- **Small learning rate:** takes small steps; convergence may be slow.
- **Suitable learning rate:** reaches the minimum efficiently.
- **Large learning rate:** may overshoot the minimum repeatedly or diverge.

For this particular function, starting from $w = 4$, the update becomes $w_{\text{new}} = (1 - 2\alpha)w$. This makes it easy to see how the learning rate changes the behavior.

3. Implement it from scratch

Run this code in your Python environment. We start every experiment from the same initial parameter so the comparison is fair.

```
import numpy as np
import matplotlib.pyplot as plt

def gradient_descent(learning_rate, initial_w=4.0, epochs=50):
    w = initial_w
    losses = []

    for epoch in range(epochs):
        loss = w ** 2
        gradient = 2 * w

        losses.append(loss)

        # Update the parameter
        w = w - learning_rate * gradient

    return losses, w

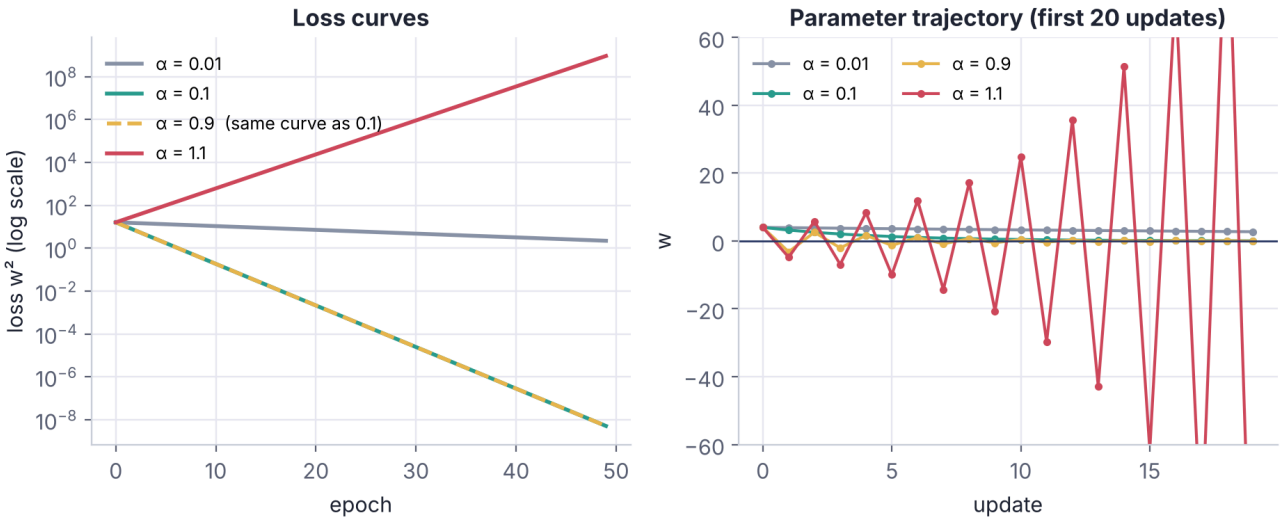
learning_rates = [0.01, 0.1, 0.9, 1.1]

plt.figure(figsize=(10, 6))

for lr in learning_rates:
    losses, final_w = gradient_descent(lr)
    plt.plot(losses, label=f"Learning rate = {lr}")

plt.xlabel("Epoch")
plt.ylabel("Loss (w²)")
plt.title("Gradient Descent: Comparing Learning Rates")
plt.legend()
plt.grid(True)
plt.show()
```

4. What should you observe?



Output of the code above. Note that α = 0.1 and α = 0.9 give identical loss curves, because |0.8| = |−0.8|.

Learning rate	Behavior	Why
0.01	Slow convergence	Each update multiplies w by 0.98. The loss decreases gradually.
0.1	Faster convergence	Each update multiplies w by 0.8, moving it toward zero much faster.
0.9	Oscillating convergence	Each update multiplies w by -0.8. The parameter jumps across zero, but its magnitude shrinks, so the loss still decreases.
1.1	Divergence	Each update multiplies w by -1.2. The parameter jumps across zero and grows in magnitude, causing the loss to increase.

For this specific loss function, gradient descent converges when $0 < \alpha < 1$. That range is **not universal** for every model or loss function.

5. Experiment with it: the learning-rate playground

In the session, an interactive playground let you change the learning rate and the number of updates. Here is its output for $\alpha = 0.01$ over 29 updates, starting from $w = 4$:

Update	w	Update	w	Update	w
0	4.000	10	3.268	20	2.670
1	3.920	11	3.203	21	2.617
2	3.842	12	3.139	22	2.565
3	3.765	13	3.076	23	2.513
4	3.689	14	3.015	24	2.463
5	3.616	15	2.954	25	2.414
6	3.543	16	2.895	26	2.366
7	3.473	17	2.837	27	2.318
8	3.403	18	2.781	28	2.272
9	3.335	19	2.725	29	2.226

Status: converges. Final parameter ≈ 2.226; final loss ≈ 4.957. Even after 29 updates, a learning rate of 0.01 is still far from the minimum at $w = 0$. Try other values with the code above (for example 0.5, 1.0 and 1.2) and look at the right-hand plot of the figure: negative values mean the parameter has crossed zero.

6. Your turn

1. Why can a learning rate of `1.1` make the loss increase continuously for this function?
2. If the learning rate is `0.5` and the initial parameter is `w = 4`, what is the parameter after one update?
3. Why isn't the smallest possible learning rate necessarily the best choice?

YOUR SUBMITTED ANSWERS

1. Because it magnifies the effect of the gradient, and it may cause overshooting and passing the optimum value.
2. $4 - 0.5 \times 2 \times 4 = 0$
3. Because with the selected number of epochs or iterations, it may never reach the optimum state because of the small steps.

7. Review of your answers

Great work! You got **3/3 conceptually correct**. Let's review each answer.

Score: 3/3.

1. Why can a learning rate of 1.1 cause divergence? CORRECT

Your answer: it magnifies the effect of the gradient, causing overshooting. **Correct.** Let's calculate what happens. Starting with $w = 4$, our update rule is:

$$w_{\text{new}} = w - \alpha(2w)$$

With $\alpha = 1.1$:

- First update: $w = 4 \rightarrow -4.8$
- Second update: $w = -4.8 \rightarrow 5.76$
- Third update: $w = 5.76 \rightarrow -6.912$

The parameter repeatedly crosses zero, but its absolute value increases. Since loss is w^2, the loss grows too.

2. Learning rate = 0.5, initial w = 4 CORRECT

Your calculation:

$$4 - 0.5 \times 2 \times 4 = 0$$

Exactly right! The gradient is 8, so the update takes us directly to the minimum. **After one update:** $w = 0$**, loss** $= 0$**.** You've reached the global minimum in a single step.

This is a special property of our simple quadratic function, not something to expect in every machine-learning problem.

3. Why isn't the smallest learning rate always best? CORRECT

Your answer is correct. A very small learning rate may require too many iterations to get sufficiently close to the minimum. For example, starting from $w = 4$:

- With $\alpha = 0.5$, we reach zero in one update.

- With $\alpha = 0.01$, after one update, $w = 3.92$, and the loss is still 15.3664.

One important nuance: a sufficiently small learning rate *can* converge eventually, given enough iterations and suitable conditions. The problem is often **how long it takes**, rather than whether it can converge at all.

8. Next challenge: a more realistic comparison

You've now learned the core mechanism. The next step is to compare learning rates on a function where the best learning rate isn't immediately obvious.

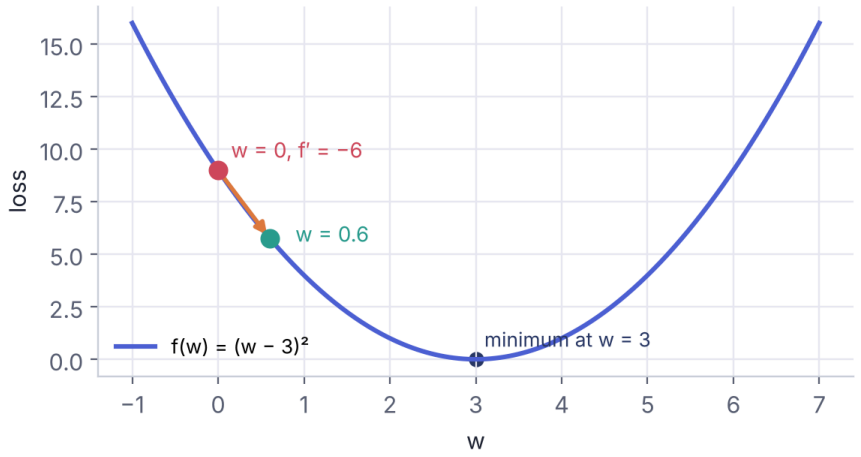
Try this: $f(w) = (w-3)^2$, starting from $w = 0$.

1. What is the gradient $f'(w)$?
2. What is the gradient at $w = 0$?
3. Using learning rate $\alpha = 0.1$, what is w after one update?

Hint: the minimum is now at $w = 3$, not at zero.

Solution *(added for this course; this challenge was not answered in the session)*

1. By the chain rule, $f'(w) = 2(w-3)$.
2. At $w = 0$: $f'(0) = 2(0-3) = -6$. The gradient is negative, so the loss decreases as w increases.
3. $w_{\text{new}} = 0 - 0.1(-6) = \boxed{0.6}$. The loss falls from $f(0) = 9$ to $f(0.6) = (0.6-3)^2 = 5.76$. Because the gradient was negative, subtracting it moved w **upward**, toward the minimum at 3.



One step on f(w) = (w − 3)²: from w = 0 to w = 0.6.

9. Common mistakes

- Confusing the loss with its gradient.
- Adding the gradient instead of subtracting it.
- Updating parameters before calculating the current gradients.
- Assuming a lower learning rate is always better.
- Assuming a fixed number of epochs guarantees convergence.
- Forgetting that the gradient depends on the current parameters and loss function.

10. Recall test: close the notes first

Try answering these from memory, then check the answers below.

1. What does the gradient tell us?
2. What happens when the learning rate is too large?
3. What is the difference between loss and gradient?
4. Why update both w and b in linear regression?
5. Does running more epochs guarantee convergence?

Answers

1. The direction of steepest increase of the loss, and how sensitive the loss is to each parameter. We step in the opposite direction.
2. Updates overshoot the minimum; training may oscillate or, if the steps are large enough, diverge with the loss growing each step.
3. The loss is a single number measuring how wrong the predictions are at the current parameters. The gradient tells us how that number changes as each parameter changes, i.e. which way and how strongly to adjust.
4. Both parameters affect the predictions and therefore the loss. Each has its own gradient: w fits the slope and b fits the intercept. Updating only one would leave the line unable to fit the data.
5. No. Convergence depends on the learning rate, the shape of the loss function, and the starting point. With a learning rate that is too large, more epochs only make divergence worse.

Your final goal: be able to explain the update rule, derive a simple gradient, implement the optimization loop in NumPy, and explain how changing the learning rate changes the loss curve, all without notes.

Module 9 · Math for Machine Learning: One-Page Cheat Sheet

Topic: Explain the fundamentals without notes · **Output:** one-page cheat sheet

Math for Machine Learning · One-Page Cheat Sheet. The essential math foundations, from data and vectors to model training and optimization.

1. Linear algebra: representing data

- **Scalar:** a single number, e.g. $x = 3$.
- **Vector:** a list of numbers representing features or parameters.
- **Matrix:** a 2D array representing multiple examples or transformations.
- **Dot product:** combines corresponding values to produce a scalar.

$$\mathbf{w} \cdot \mathbf{x} = \sum_{j=1}^{d} w_j x_j$$

Linear regression uses this to combine features into a prediction:

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

Here, $\mathbf{x}$ contains features, $\mathbf{w}$ their weights, and b the bias.

2. Functions: mapping inputs to outputs

A function describes how one quantity depends on another. A model is a function with adjustable parameters, e.g. $f(x) = x^2$. In ML, a model $f_\theta(x)$ maps input data to predictions; θ represents its trainable parameters.

3. Derivatives: measuring change

A derivative tells you how quickly a function changes as an input changes: $\frac{d}{dx}x^2 = 2x$. At $x = 3$, the slope is 6. The derivative is positive, so increasing x locally increases the function.

Partial derivatives measure change with respect to one parameter while holding the others fixed. The **gradient** collects these derivatives:

$$\nabla L(\theta) = \begin{bmatrix} \frac{\partial L}{\partial \theta_1} \\ \vdots \\ \frac{\partial L}{\partial \theta_d} \end{bmatrix}$$

4. Probability and statistics: understanding data

- **Mean:** average value.
- **Variance:** average squared deviation from the mean.
- **Standard deviation:** square root of variance.
- **Probability:** quantifies uncertainty.
- **Conditional probability:** $P(A \mid B)$, probability of A given B.
- **Bayes' theorem:** updates a probability using evidence.

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

These concepts help describe data distributions, uncertainty, and statistical predictions.

5. Loss functions: measuring model error

A loss function quantifies how wrong a model's predictions are. Training aims to minimize it.

Mean squared error (regression):

$$L = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

Binary cross-entropy (binary classification):

$$L = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log p_i + (1 - y_i)\log(1 - p_i)]$$

Lower loss generally means better fit to the training targets, but does not guarantee better performance on unseen data.

6. Optimization: learning model parameters

Gradient descent adjusts parameters in the direction that locally reduces the loss:

$$\theta_{\text{new}} = \theta - \alpha \nabla L(\theta)$$

- θ: model parameters.
- ∇L: gradient of the loss.
- α: learning rate, controlling step size.

Too small: slow progress. **Too large:** possible overshooting or divergence.

7. The complete ML training cycle

Data and features → model makes predictions → loss measures error → gradients measure parameter sensitivity → optimizer updates parameters → repeat, then evaluate on unseen data.

The big picture

- **Linear algebra** represents data and model parameters.
- **Functions** describe the model's predictions.
- **Calculus** tells us how the loss changes when parameters change.
- **Probability and statistics** help us reason about uncertainty and data.
- **Loss functions** define what the model should improve.
- **Optimization** uses gradients to improve the model.

This is the core mathematical foundation for understanding linear regression, logistic regression, neural networks, and eventually transformers.